Planning Fast and Slow: Trajectory Synthesis via Conditional Flow Matching

Frank Tsu-Fang Lu

Master of Science in Computer Vision, Robotics and Machine Learning

from the

University of Surrey

School of Computer Science and Electrical and Electronic Engineering

Faculty of Engineering and Physical Sciences

University of Surrey

Guildford, Surrey, GU2 7XH, UK

September 2025

Supervised by: Dr. Simon Hadfield

DECLARATION OF ORIGINALITY

I confirm that the project dissertation I am submitting is entirely my own work and that any material used from other sources has been clearly identified and properly acknowledged and referenced. In submitting this final version of my report to the JISC anti-plagiarism software resource, I confirm that my work does not contravene the university regulations on plagiarism as described in the Student Handbook. In so doing I also acknowledge that I may be held to account for any particular instances of uncited work detected by the JISC anti-plagiarism software, or as may be found by the project examiner or project organiser. I also understand that if an allegation of plagiarism is upheld via an Academic Misconduct Hearing, then I may forfeit any credit for this module or a more severe penalty may be agreed.

MSc Dissertation Title: Guided Trajectory Synthesis via Flow Matching

Author Name: Frank Tsu-Fang Lu

Author Signature:  Date: September 2, 2025

Supervisor's name: Simon Hadfield

WORD COUNT

Number of Pages: 83

Number of Words: 16109

ABSTRACT

Generative modeling has emerged as a powerful paradigm for offline reinforcement learning (RL), with diffusion models showing particular promise for synthesizing complex, multimodal trajectories. However, a fundamental tension exists between their iterative, stochastic nature and the stringent low-latency and deterministic requirements of real-world robotic control. This conflict presents a significant barrier to deploying such models in time-critical, physical systems.

This dissertation introduces Flow Planner, a novel hierarchical framework built upon Conditional Flow Matching (CFM)—a deterministic and computationally efficient alternative to diffusion models. The framework adapts CFM for state–action sequence generation by decomposing the planning problem into two complementary modules: a long-horizon Waypoint Planner for strategic reasoning and a short-horizon Action Planner for reactive control. These planners are seamlessly integrated within a receding-horizon model predictive control (MPC) loop, enabling a unique blend of deliberative and reactive behavior.

Extensive experiments on the Gymnasium LunarLander benchmark demonstrate that Flow Planner achieves substantially faster inference speeds while maintaining competitive, and often more stable, control performance compared to leading diffusion-based and monolithic baselines. In addition to these performance gains, this work contributes a reproducible data curation pipeline and a benchmark dataset with a trained expert agent to support future research. These results establish flow matching as a robust foundation for efficient, modular, and scalable generative planning. The Flow Planner framework offers a principled approach to bridging the gap between offline RL theory and its practical application in real-time robotic systems.

CONTENTS

LIST OF FIGURES

1 INTRODUCTION

Imbuing intelligent agents with the capability for autonomous-decision-making in real-world environments is challenging. For decades, reinforcement learning has offered a robust mathematical framework for acquiring optimal behavioral skills through interactions within an environment to maximize cumulative rewards. The integration of deep neural networks as high-capacity function approximators has propelled RL even farther, leading to remarkable achievements across diverse domains, from mastering complex games to enabling basic robotic locomotion and manipulation. However, the widespread success of modern machine learning, as seen in areas such as computer vision and natural language processing, fundamentally hinges on the availability of large and highly diverse datasets. As seen in Figure 1.1, in conventional, online RL, this data is collected through a continuous interaction loop: the agent executes its current policy in the environment, collects the resulting experiences into a buffer, and then uses this fresh data to update its policy. This cycle of exploration and optimization is repeated until the policy converges. With this paradigm, the acquisition of such extensive data becomes prohibitively impractical for real-world applications. For example, autonomous driving would need to practice in numerous cities for every model update, which is costly, time-consuming, let alone being unsafe. This inherent limitation created a gulf of generalization between lab-bound RL successes and the complex, open-world settings where intelligent decision-making is most critically needed.

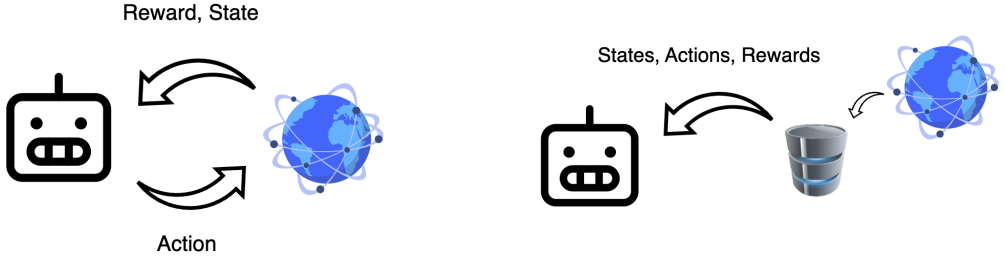

Figure 1.1: Comparison of Online and Offline Reinforcement Learning.

Offline Reinforcement Learning emerges as a pivotal paradigm to bridge this gap. Offline RL focuses on enabling the training of policies solely from previously collected

datasets, without additional online interaction. This approach holds immense potential for transforming vast amounts of historical data into robust, generalizable decision-making engines, much like how supervised learning revolutionized pattern recognition. The implications are profound across various fields from robotics, healthcare, and autonomous driving to logistics, finance, and operations research, as it offers a viable pathway to deploy sophisticated AI systems in domains where active data collection is unfeasible, costly, or unsafe.

Finally, the ability to leverage large and diverse datasets in an offline setting aligns the field of robotics learning with the broader paradigm of foundation models, as seen in computer vision and natural language processing. This opens the door to pre-training generalist agents on massive, heterogeneous datasets to acquire a broad repertoire of skills, which can then be efficiently fine-tuned for specific downstream tasks. This paradigm thus offers a compelling vision for the future: transforming vast archives of passive, observational data into active, embodied intelligence.

1.1 Background and Context

Despite its immense promise, offline RL is plagued by algorithmic challenges. At its core, offline RL is fundamentally about making and answering what Levine et al. describe as **"counterfactual queries"** [1]. These are "what if" questions about what might happen if the agent were to carry out actions different from those in the dataset. In other words, for an agent to improve upon the underlying behavior policy in the offline dataset, it must be able to execute action sequences that diverge from the dataset. This inherent need for counterfactual inference strains conventional machine learning tools, which assume that test data is drawn from the same distribution as training data, leading to a problematic distribution shift.

This issue is particularly acute for value-based methods. These methods learn a function to estimate the expected reward, or "goodness", of an action. When queried about an action that is far from the training data, this value estimate has no reliable data to ground its prediction. Function approximation errors can then cause the model to become unreliably optimistic, assigning erroneously high values to these unsupported actions [1]. The policy, trusting these flawed estimates, is then misguided into choosing brittle and unsafe behaviors when deployed.

This instability is particularly exacerbated by high-dimensional function approximators such as Deep Neural Networks (DNNs). The errors can accumulate over long horizons, making stable learning notoriously difficult. Furthermore, real-world datasets, especially those from human demonstrations, inherently exhibit multimodal action distributions: there are often multiple valid ways to perform a task. Traditional methods like behavioral cloning struggle to capture this multimodality, often averaging across modes and leading to indecisive or jerky behaviors that fail to generalize [2].

To address the challenge of multimodality, generative modeling, particularly with denoising diffusion models, has emerged as a powerful approach [3]. By framing planning as a sampling process, diffusion models can learn and represent complex, multi-modal distributions directly from data. This is a critical capability for robotics, where a task may have multiple valid solutions, such as different grasps for an object or various paths around an obstacle. By capturing the entire distribution of expert solutions, these models can enhance a robot's versatility and ability to generalize [2, 4].

However, the very properties that make diffusion models powerful for generation introduce fundamental challenges for their deployment in real-time robotic systems. These models are built upon a stochastic, iterative framework that imposes a significant computational burden, leading to high-latency inference incompatible with dynamic control loops. More importantly, their stochastic nature is fundamentally misaligned with the deterministic physical laws and hard safety constraints that govern robotic motion, calling for an alternative generative paradigm.

Therefore, this dissertation presents Flow Planner, a hierarchical offline RL framework built upon flow matching, a powerful alternative that directly addresses the core limitations of diffusion models [5]. Flow matching learns a deterministic transformation governed by an Ordinary Differential Equation (ODE) rather than a Stochastic Differential Equation (SDE) [6]. This fundamental distinction makes it not merely an incremental improvement but a paradigm inherently aligned with the deterministic, low-latency requirements of physical robotic systems.

1.2 Objectives

The primary aim of this dissertation is to develop and validate a novel, modular framework for imitation learning in offline reinforcement learning, leveraging the principles of flow matching for a high-speed alternative to contemporary diffusion-based planners. The specific objectives established to achieve this aim are as follows:

1. **To develop a specialized Conditional Flow Matching (CFM) formulation for trajectory planning** that efficiently generates coherent state-action sequences for robotic control, adapting techniques previously used in image and video generation.
2. **To architect a hierarchical control framework** using flow-based planning that separates strategic decision-making from tactical execution, effectively solving the scalability and horizon limitations of end-to-end approaches.
3. **To empirically validate the framework** through rigorous benchmarking against monolithic planners and state-of-the-art diffusion models, focusing on control effectiveness, inference speed, and trajectory quality.
4. **To design and implement a reproducible data curation pipeline** for the training and validation of the proposed generative planners, ensuring the work is transparent and reproducible by future research.

1.3 Achievements

The primary aim of this dissertation is to develop and validate a novel, modular framework for imitation learning in offline reinforcement learning, leveraging the principles of flow matching for a high-speed alternative to contemporary diffusion-based planners. The specific objectives established to achieve this aim are as follows:

1. **A Novel Formulation of Flow-Based Trajectory Planning**: This work introduces the first specialization of the Conditional Flow Matching (CFM) framework for the domain of robotic trajectory planning. By reformulating the CFM objective to generate different sequences of states and actions, this contribution establishes a new class of deterministic, high-speed generative planners that directly address the latency limitations of contemporary diffusion-based methods.

2. **A Hierarchical Framework for Long-Horizon Control**: The second key contribution is the Flow Planner architecture, a new two-stage hierarchical framework that decouples long-horizon strategic reasoning from low-level reactive control. This design solves the scaling and coherence problems inherent in monolithic, end-to-end planners by factorizing the planning problem into a strategic Waypoint Planner and a reactive Action Planner, a novel application for flow-based models.

These core contributions are supported by:

1. **A Comprehensive Analysis of Flow-Based Planners**: This work provides an in-depth empirical analysis of applying flow-matching models to robotics, offering insights into the role of different model architectures and conditioning mechanisms, alongside a direct comparison to the diffusion paradigm.
2. **A Reproducible Data Curation Pipeline and Benchmark Datasets**: This work contributes a flexible and extensible pipeline for curating high-quality offline reinforcement learning datasets. This pipeline was used to generate three novel benchmark datasets, which have been publicly released alongside their corresponding trained expert agents to facilitate reproducibility and future research.

These contributions culminate in a robust and efficient framework for imitation learning, with the findings currently being prepared for submission to the IEEE International Conference on Intelligent Robots and Systems (IROS).

1.4 Overview of Dissertation

The remainder of this dissertation is structured as follows:

- **Chapter 2** provides the theoretical foundation for this work, first examining established methods for addressing distribution shift problems, then exploring contemporary solutions that leverage generative models.
- **Chapter 3** presents the core technical foundation of this dissertation. It details various approaches to dataset modeling with comprehensive mathematical formulations.

The chapter ends by introducing "Flow Planner," the proposed hierarchical planning framework, and provides implementation specifics using Meta's Flow Matching library [7].

- **Chapter 4** details the methodology for data curation and introduces the experimental environments. This includes a technical discussion on the implementation of the data pipeline using Gymnasium[8] wrappers and the Minari[9] library for dataset management.
- **Chapter 5** provides a comprehensive analysis and discussion of the empirical results. This includes an investigation of various conditioning mechanisms and a benchmark comparison of the framework's performance against baseline models.
- **Chapter 6** concludes the dissertation by summarizing the key contributions, discussing the implications of the findings, and outlining several promising directions for future research.

2 BACKGROUND THEORY AND LITERATURE REVIEW

This chapter begins by examining the fundamental concepts in offline reinforcement learning and established methodologies for addressing distributional shift. Next, it explores diffusion models through the lens of variational inference before unifying these concepts through continuous flows to introduce flow matching. The chapter concludes with an analysis of diffusion applications in trajectory synthesis and state-of-the-art approaches that integrate these models directly into reinforcement learning frameworks. Particular emphasis is placed on the mathematical formulations of diffusion and flow matching, as this constitutes the central focus of this dissertation and the applied methodologies after.

2.1 Offline Reinforcement Learning

As established in Chapter 1, a significant hurdle in offline reinforcement learning is the distributional shift between the static dataset and the policy being learned. This can lead to erroneous value estimates for out-of-distribution actions, resulting in brittle policies that perform poorly when deployed [1]. To mitigate this issue, the literature has broadly converged on three primary classes of approach: policy constraint methods, implicit policy constraint methods, and model-based methods.

2.1.1 Policy Constraint Methods

One early class of methods that's been explored extensively is policy constraint methods. These approaches aim to mitigate distributional shift by updating the policy not just by maximizing the Q-value, but by doing so subject to a constraint that the learned policy remains sufficiently close to the behavioral policy (collected data). This closeness is typically measured by a divergence metric, such as KL divergence or Maximum Mean Discrepancy (MMD).

To a degree, these methods can bound the distributional shift and the impact of out-of-distribution actions, potentially leading to more stable learning. Examples include BEAR (Bootstrapping Error Accumulation Reduction) by Kumar et al. [10] and BRAC (Behavior Regularized Actor Critic) by Wu et al. [11].

A significant challenge is their potential to be overly conservative, as a single constraint value ϵ might not work well across all states, especially if the behavior policy exhibits both highly random and highly deterministic actions. More critically, these methods often require accurately estimating the behavior policy itself, which can be extremely difficult, particularly when the data comes from diverse sources (e.g., humans or a mixture of different policies). Errors in this estimation can lead to an imperfect constraint and poor practical performance, making it a major bottleneck, especially during online fine-tuning.

2.1.2 Implicit Policy Constraint Methods

Building upon the insights into the limitations of explicit policy constraint methods, implicit policy constraint methods, such as AWAC (Advantage Weighted Actor Critic) by Nair et al. [12], emerged as an alternative that addresses the difficulty of behavior policy modeling. AWAC leverages a theoretical identity that shows the solution to a constrained optimization problem can be approximated using a weighted maximum likelihood objective. In this formulation, actions taken from the behavior policy are weighted by the exponentiated advantage function, effectively embedding the constraint implicitly [10]. More intuitively, AWAC approximates the best new policy by look at actions already in the dataset and give more weight (advantage) to the really good ones.

This approach obviates the need for explicit modeling of the behavior policy, only requiring samples from it, which are readily available in the offline dataset. AWAC has shown to be effective for online fine-tuning after offline initialization and demonstrates strong performance on complex tasks like dexterous manipulation with human demonstration data.

While the sources highlight its advantages, it is still a form of policy constraint. Its inherent conservativeness might limit the extent to which it can discover entirely novel, optimal behaviors far from the observed data.

2.1.3 Model-based Offline RL Methods

Taking a different algorithmic direction, model-based offline RL methods, exemplified by MOPO (Model-based Offline Policy Optimization) by Yu et al. [13], propose to tackle the problem by learning a model of the environment's dynamics. While vanilla model-based RL (like MBPO [14]) already showed some promise in offline settings, MOPO specifically

modifies these algorithms to handle distributional shift. MOPO achieves this by punishing the policy for exploiting out-of-distribution states during model rollouts. This is implemented by subtracting an uncertainty penalty from the reward function, which becomes larger for state-action tuples that are more out of distribution (i.e., where the learned model's predictions are less certain).

MOPO hence has the ability to generalize beyond the state and action support of the offline data, enabling the learning of policies for tasks different from those for which the data was collected. It is more resilient to overestimation and overfitting than model-free methods and theoretically maximizes a lower bound of the policy's true return, balancing exploration and risk. Empirically, MOPO outperforms existing model-free offline RL methods and vanilla model-based approaches, especially on tasks requiring out-of-distribution generalization.

Nevertheless, models can still be inaccurate in regions far from the training data, and in a purely offline setting, these errors cannot be corrected through further interaction. The effectiveness relies heavily on the quality of the uncertainty estimation and the tuning of the penalty coefficient [1]. It may also struggle with datasets lacking sufficient action diversity, and there are limits to its generalization capabilities if the new task requires exploring states completely outside the available data support.

2.1.4 Conservative Q Learning

Finally, a distinct and highly effective approach is Conservative Q-Learning (CQL) by Kumar et al. [15], which directly addresses the core problem of Q-value overestimation. Instead of constraining the policy, CQL augments the standard Bellman error objective with a novel Q-value regularizer. This regularizer works by finding and minimizing (pushing down) erroneously high Q-values for out-of-distribution actions, while a more advanced version also compensates by pushing up Q-values for actions present in the dataset. This makes the learned Q-function inherently "pessimistic".

CQL is theoretically guaranteed to learn a lower bound on the true Q-function, effectively preventing the problematic overestimation. This direct approach leads to state-of-the-art performance across a wide range of tasks on standard benchmarks like D4RL[16] and Atari[8]. It significantly outperforms many previous offline RL methods, demonstrat-

ing the ability to ”stitch” together parts of different trajectories to find more optimal paths, and achieving up to five times better results on challenging dexterous manipulation tasks. While conservative, the full CQL method only slightly underestimates Q-values, making it a robust and reliable algorithm.

The effectiveness of CQL depends on the appropriate selection of the regularization weight α. Simpler variants of the algorithm without the compensatory term for in-distribution actions can lead to excessive underestimation. Despite its strong performance, ongoing research continues to explore further improvements.

In summary, policy constraint methods and implicit policy constraint methods both try to constrain the learned policy to remain close to the behavior policy. However, they do not aim to learn a pessimistic Q-function or ensure a lower bound on Q-values. They simply avoid the problematic evaluation of out-of-distribution actions. On the other hand, MOPO and CQL are methods that embrace a conservative and “pessimistic” strategy in order to mitigate the risks associated with going out of distributions and value overestimation by either directly pushing down the Q-values or penalized rewards based on uncertainty. This ensures the learned policies are robust, accepting the trade-off that may lead to sub-optimal but reliable performance.

2.2 Diffusion Models

The following two sections provide the necessary background theory on diffusion and score-based models preceding flow matching. Section 2.2 introduces the formulation of diffusion models, one derived from a variational inference perspective. Section 2.3 explores a more general score-based approach defined by continuous-time stochastic differential equations. Finally, section 2.4 introduces flow matching, a simulation-free method for training Continuous Normalizing Flows based on deterministic Ordinary Differential Equations (ODEs).

2.2.1 Diffusion’s Roots in Variational Bayesian Methods

In probabilistic generative modeling, the common practice is to introduce latent variables z and a parameterized generative model $p_\theta(x)$ with prior $p(z)$ to explain observations x. By Bayes’ rule, the model’s posterior distribution is expressed as:

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)}. \tag{2.1}$$

Where the denominator is the marginal likelihood or evidence, calculated by 2.2:

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz. \tag{2.2}$$

The posterior $p(z \mid x)$ explains how likely different latent configurations are given the data, but computing it directly is intractable because calculating the evidence requires integrating over latent space, which explodes in complexity. This intractability means we cannot directly evaluate or differentiate $p(x)$.

Hence, the Bayesian objective is maximizing the marginal likelihood, or evidence of the observed data $\log p_\theta(x)$. It quantifies how well a generative model explains the data overall, marginalizing out the latent space. Variational Bayesian methods provide a practical workaround by introducing a tractable approximation $q_\phi(z \mid x)$, referred to as a recognition model or probabilistic encoder, to the true posterior and maximizing the evidence lower bound (ELBO)[17]:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x, z) - \log q_\phi(z \mid x)\Big], \tag{2.3}$$

or more explicitly shown as:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log p_\theta(x \mid z) + \log p(z) - \log q_\phi(z \mid x)\Big]. \tag{2.4}$$

2.2.2 Deep Unsupervised Learning using Nonequilibrium Thermodynamics

Building on principles from non-equilibrium thermodynamics, diffusion probabilistic models systematically and slowly destroy data structure by applying an iterative forward diffusion process that gradually perturbs data $x_0 \sim p_{\text{data}}(x)$ into a tractable Gaussian noise via a Markov chain:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}). \tag{2.5}$$

This interestingly mirrors physical diffusion processes such as ink dispersing in water or smoke spreading in air. This forward process spreads probability mass across the data landscape, creating training signals far from the original data and grounding the method in the same mathematics as Brownian motion.

A reverse diffusion process is then learned to restore this structure. The ideal reverse process $q(x_{t-1} \mid x_t)$, as mentioned in 2.2.1, is intractable as it depends on the entire data distribution. Dickstein et al. therefore parameterized an approximation p_θ designed to match the tractable posterior $q(x_{t-1} \mid x_t, x_0)$ [18]:

$$p_\theta(x_{t-1} \mid x_t) \;\approx\; q(x_{t-1} \mid x_t, x_0). \tag{2.6}$$

The trained model serves as a local compass that estimates the gradient of the log probability landscape $\nabla_x \log p_t(x)$, which points towards regions of higher data likelihood. Here, $p_t(x)$ denotes the time-varying marginal distribution of data at diffusion step t, obtained by evolving clean samples $x_0 \sim p_{data}$ through the forward noising process. This model does not learn $p_t(x)$ directly, but instead estimate its score function. This connects diffusion to score matching later covered extensively in Section 2.3.

Adapting variational inference, one can derive a variational lower bound on this log-likelihood:

$$\log p_\theta(x_0) \;\geq\; \mathbb{E}_{q(x_{0:T})}\Big[\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0)\Big]. \tag{2.7}$$

This entire framework can therefore be understood intuitively through the lens of variational inference:

- The fixed forward noising process $q(x_{1:T} \mid x_0)$ serves as the probabilistic encoder, mapping the data x_0 into a latent space of noisy variables $x_{1:T}$.
- The learned reverse process $p_\theta(x_{t-1} \mid x_t)$ acts as the probabilistic decoder, which is trained to reverse the noising process.

This means that training diffusion models amounts to optimizing a diffusion-specific ELBO, which remarkably simplifies to training the decoder to predict the noise added at each step, as demonstrated in the subsequent section.

2.2.3 Denoising Diffusion Probabilistic Models

While earlier works established the connection between diffusion and variational inference, the objective was still complex. Ho et al.'s significantly popularized the framework by reparameterizing the model's learning target [3]. The reverse process learns the transition

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1};\, \mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)\big), \tag{2.8}$$

which is modeled as a Gaussian with a mean $\mu_\theta(x_t, t)$ and variance $\Sigma_\theta(x_t, t)$. The DDPM authors made two key simplifications: First, they found that fixing the variance to a known constant was sufficient for high-quality results. Second, and most importantly, they reparameterized the mean. Instead of training the network to directly predict the mean of the reverse step, they trained it to predict the noise, ϵ, that was originally added to the clean data. This transforms the complex variational objective into a simple mean-squared error loss between the true noise and the predicted noise:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \epsilon}\big[\, \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \,\big], \tag{2.9}$$

where ϵ is the true Gaussian noise sampled at a random timestep t, and $\epsilon_\theta(x_t, t)$ is the network's prediction given the noisy image x_t and timestep t. This simplified objective was not only more stable to train but also produced significantly higher-quality samples, which was the major breakthrough that led to the widespread adoption of diffusion models.

2.3 Score-based Models

An alternative and powerful approach to generative modeling is to learn the score function of the data distribution, which is formally defined as the gradient of its log probability $\nabla_x \log p_{\text{data}}(x)$ [19]. This score function creates a vector field that points towards regions of higher data density, providing a trajectory back to the data manifold from any point in the space. Once this score field is learned, new samples can be generated by starting

from random noise and iteratively moving in the direction of the score, a process known as Langevin dynamics.

2.3.1 Denoising Score Matching

Estimating the score function directly is difficult. Vincent et al. provided a workaround in Denoising Score Matching (DSM) [19]. The key insight was that the objective of training a simple network to denoise data corrupted with Gaussian noise is equivalent to learning the score of that noise-perturbed data distribution. Given a clean sample x, corruption is defined as

$$\tilde{x} = x + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \tag{2.10}$$

$\tilde{x}$ being a noisy sample. When the corruption is additive isotropic Gaussian noise, the target score $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)$ for a noisy sample $\tilde{x}$ given a clean sample x simplifies to $\frac{1}{\sigma^2}(x - \tilde{x})$.

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = \frac{1}{\sigma^2}(x - \tilde{x}) = -\frac{\epsilon}{\sigma}. \tag{2.11}$$

This shows that the target score is directly proportional to the negative noise. This is why instead of a complex score matching objective, one could simply train a denoising network $D_\theta(\tilde{x})$ with a straightforward mean-squared error loss like the one in DDPM:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{x,\tilde{x}}\Big[\, \|D_\theta(\tilde{x}) - x\|^2 \,\Big]. \tag{2.12}$$

This made score estimation practical and stable.

2.3.2 Generative Modeling by Estimating Gradients of the Data Distribution

While DSM worked for a single noise level, it struggled with real-world data that often resides on a low-dimensional manifold, leaving the score undefined in most of the space. Song et al. solved this problem similar to diffusion by perturbing the data with multiple levels of Gaussian noise [20]. They then trained a single Noise Conditional Score Network (NCSN), denoted $s_\theta(x_t, t)$, to jointly estimate the scores for all the different

noise-perturbed distributions. This effectively creates a well-defined vector field at all locations and scales.

For generation, annealed Langevin dynamics is introduced. The process starts with pure Gaussian noise and follows the gradient provided by the learned score network s_θ. The noise level σ is slowly annealed from high to low, guiding the sample from the simple noise distribution back to the complex data distribution via an iterative update step:

$$x_{t-1} = x_t + \alpha s_\theta(x_t, \sigma_t) + \sqrt{2\alpha} z_t \tag{2.13}$$

where α is the step size and $z_t \sim \mathcal{N}(0, I)$ is a standard Gaussian noise term.

2.3.3 Score-Based Generative Modeling Through Stochastic Differential Equations

The SDE framework unifies score matching and DDPMs by describing them as discretizations of a single continuous-time process [21].

The forward SDE is defined to transform a complex data distribution into a simple noise prior over a time interval $t \in [0, T]$:

$$dx = \underbrace{f(x,t)}_{\text{drift}}\, dt + \underbrace{g(x,t)}_{\text{diffusion}}\, dW, \tag{2.14}$$

where W is a standard Wiener process and $f(x,t)$ and $g(x,t)$ are the drift and diffusion coefficients, respectively. This describes a continuum of noisy intermediate distributions $\{p_t(x)\}$. Additionally, a corresponding reverse-time SDE that can reverse this noising process exists:

$$dx = \underbrace{\left[f(x,t) - g(x,t)^2 \nabla_x \log p_t(x)\right]}_{\text{drift}}\, dt + \underbrace{g(x,t)}_{\text{diffusion}}\, d\bar{W}, \tag{2.15}$$

where $d\bar{W}$ is a standard Wiener process run backward in time. Much like the forward SDE, one can intuitively break this equation down into a deterministic drift term and a stochastic diffusion term. And conveniently, the deterministic dynamics depend only on the score function $\nabla_x \log p_t(x)$. Note that for models like DDPMs, the diffusion term simplifies from the general form $g(x,t)$ to $g(t)$ because the noise schedule is designed to be state-independent, as detailed further in A.1. Since this score is intractable, it is approximated

by a neural network $s_\theta(x, t)$, which can then be used by numerical solvers to generate samples [6]. This mathematically elegant framework unlocks some new capabilities:

1. Flexible sampling: Because the process is continuous, practitioners are no longer locked into a fixed number of discrete steps. This allows for a direct trade-off between computation speed and sample quality, as any numerical SDE solver can be used to generate samples.

2. The continuous nature of the SDE makes it highly amenable to controllable generation. The reverse sampling process can be guided to solve inverse problems like image inpainting and colorization, often without needing to retrain the model.

3. Exact Log-Likelihoods via an Equivalent ODE: The SDE framework revealed a corresponding deterministic process, described by a neural ordinary differential equation (ODE) known as the probability flow ODE [22]. This breakthrough not only enables deterministic sampling and exact likelihood computation but also provides the direct theoretical inspiration for Flow Matching, which will be detailed in the next section.

2.4 Flow-based Models

Sections 2.2 and 2.3 laid the foundation of simulation-based models, where generating a sample requires simulating a multi-step denoising process. This section now turns to a powerful successor paradigm: the recently proposed flow-based models. Representing the next generation of generative modeling, this framework learns a direct, invertible transformation that can map noise to data in a single, continuous pass. By reframing the generative process around deterministic Ordinary Differential Equations (ODEs), this approach offers substantial advantages, most notably a dramatic reduction in inference time, a key motivation for the planning framework developed in this dissertation.

2.4.1 Normalizing Flows

The core idea of Normalizing Flows is to learn an invertible function f that warps a simple base distribution like a standard Gaussian $p_z(z)$ to a data distribution $p_X(x)$ directly [22]. Using the change of variables formula, the probability density of $x = f(z)$ can be calculated exactly:

$$p_X(x) = p_Z(f^{-1}(x)) \cdot \left| \det \left(\frac{\partial f^{-1}(x)}{\partial x} \right) \right|. \tag{2.16}$$

The $\frac{\partial f^{-1}(x)}{\partial x}$ is the Jacobian matrix of the inverse transformation, and its determinant measures how the function locally stretches or compresses space. For a general, unrestricted neural network, this determinant is computationally infeasible as data dimension scales up.

To solve this, NFs decompose f into a series of transformations $f_1 \circ \cdots \circ f_k$, each designed to have a Jacobian determinant that is easy and efficient to compute. However, this poses architectural constraints as the network cannot be a dense web of all-to-all connections. Specialized blocks like coupling layers are used in RealNVP[23] and Glow [24]. These layers operate by splitting the input data in half and leaving one half frozen while transforming the other. Though this makes training feasible, the necessary rigidity limits the model's expressivity and flexibility.

2.4.2 Continuous Normalizing Flows

The architectural limitations of discrete NFs motivated Continuous Normalizing Flows (CNFs). Instead of a finite stack of layers, a CNF conceptualizes the transformation as a continuous-time process using Ordinary Differential Equations (ODEs) [25]. The entire process can be visualized as a smooth "flow" that gradually warps a simple initial distribution into a complex target data distribution, as illustrated in the figures 2.1.

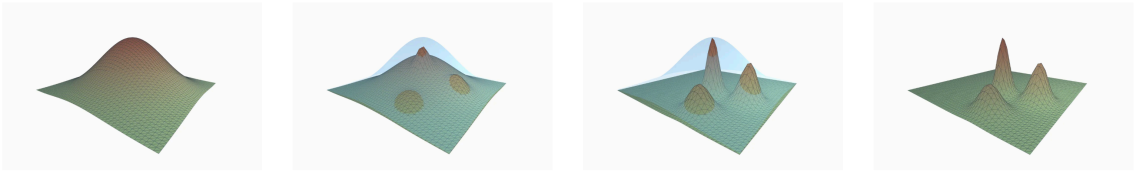

Figure 2.1: An illustration of a Continuous Normalizing Flow. Over time, a simple, unimodal Gaussian distribution (left) is progressively transformed by a learned vector field into a complex, multimodal data distribution (right).

In other words, instead of having the neural network representing the entire transformation, it defines a time-dependent vector field $v(z(t), t, \theta)$. This vector field dictates the direction and speed that any point z moves at any given time t. The dynamics of this transformation can be expressed as:

$$\frac{dz(t)}{dt} = v(z(t), t, \theta). \tag{2.17}$$

This continuous formulation elegantly sidesteps the primary issue of discrete NFs. The change of variables formula from 2.16 now becomes:

$$log(p_X(x)) = log(p_Z(z(0))) - \int_0^1 Tr\left(\frac{\partial v(z(t), t, \theta)}{\partial z(t)}\right) dt. \tag{2.18}$$

Since the trace of a Jacobian is much more efficient to compute than the full determinant, there are no architectural constraints. Any standard neural network architecture like a ResNet [26] can be used to learn the vector field, granting CNF much more flexibility. However, to calculate the likelihood for each training step, a numerical ODE solver must compute the entire trajectory of a data point. Furthermore, computing gradients requires solving a second adjoint ODE. This is computationally and memory intensive, as well as being numerically unstable.

2.4.3 Flow Matching for Generative Models

Flow Matching was introduced to bypass the training bottleneck of CNF by reframing the CNF training objective from complex likelihood computations into a direct regression objective [5].

To construct such an objective, a continuous probability path $p_t(x)$ is first defined. This path smoothly interpolates between a simple prior distribution $p_0(x)$ at $t = 0$ and the data distribution $p_1(x)$ at $t = 1$. The key insight is that any such path has a unique corresponding vector field $u_t(x)$ that generates it. This relationship is governed by the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot [p_t(x) u_t(x)]. \tag{2.19}$$

This equation provides a link between the rate of change of the probability density and the divergence $\nabla$of the probability flux. The divergence measures how much the vector field is "flowing out" of a given point. In essence, the continuity equation formalizes the intuition that if more probability flows away from a region than into it, the density in that region must decrease. This principle then guarantees the existence of a "ground truth" vector field u_t for any defined path p_t. The goal of flow matching is to train a neural network $v(x, t, \theta)$ to learn this underlying vector field, which can be visualized as a time-varying "current" that guides the probability mass, as shown in figure 2.2

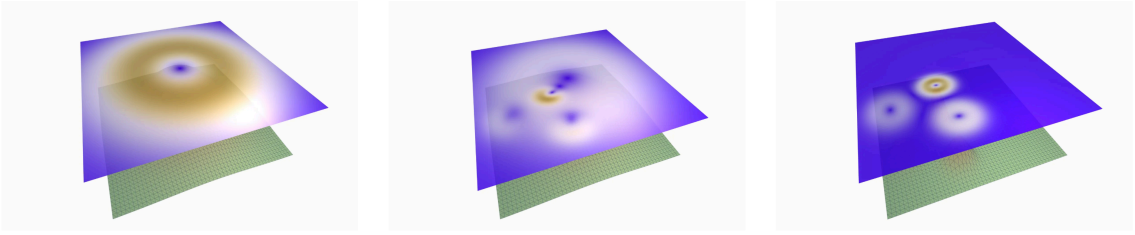

Figure 2.2: A visualization of the learned vector field at different times t. The field represents the direction and magnitude of the "flow" that transports the initial noise distribution (left) towards the target data distribution (right).

The general flow matching objective then becomes a straightforward L2 loss between the model's predicted vector field and the target vector field:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,p_t(x)}\left[\|v(x,t,\theta) - u_t(x)\|^2\right]. \tag{2.20}$$

This objective is simulation free because it only requires sampling a time t and a point x from the known marginal distribution $p_t(x)$ and regressing the model's output onto the target vector [6]. While this general formulation is theoretically convenient, it requires access to both the marginal probability path $p_t(x)$ and the marginal vector field $u_t(x)$. For most paths connecting a simple prior to a complex data distribution, both of these quantities are intractable. This issue is the primary motivation for conditional flow matching, with a practical implementation covered extensively in Section **??** that constructs the path in a specific way to ensure the target vector field is simple and known.

2.5 Planning with Diffusion

Section 2.1 highlighted a core challenge in offline reinforcement learning: calibrating the precise degree of pessimism to prevent value overestimation without creating overly conservative policies. The generative models detailed in Sections 2.2, 2.3 and 2.4 offer a powerful framework for solving this issue.

Among these, diffusion models have been successfully applied to practical trajectory planning problems [2, 4]. Rather than relying on explicit uncertainty penalties, diffusion models learn the underlying behavioral policy itself. These trained models function as a high-fidelity sampler for in-distribution actions, providing a powerful yet implicit constraint on the learned policy. This solves the pessimism calibration problem by ensuring

the policy does not stray into unsupported regions of the state-action space. This section now reviews key papers that use this approach for trajectory planning.

2.5.1 Flexible Behavior Synthesis using Diffusion

Model-based methods like MOPO, as referenced in 2.1.3, rely on training a dynamics model to predict future states, but unrolling these learned dynamics models over long horizons often results in compounding prediction errors and leads to out-of-distribution states. This in turn generates adversarial plans that appear optimal but yield poor rewards in practice. Janner et al. addresses these by reconceptualizing planning as a generative modeling problem, training a diffusion model to directly generate entire trajectories [4]. This approach offloads reward optimization and dynamics rollouts into the generative process itself, making planning a sampling problem where guidance can be used to construct desired plans.

The proposed model, Diffuser, operates by a non-autoregressive prediction procedure, generating the entire trajectory simultaneously from Gaussian noise. A trajectory τ is represented as an interleaved sequence of states $s$and actions a of a finite horizon H:

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_H, a_H) \tag{2.21}$$

A fixed forward process $q(\tau^i|\tau^{i-1})$ gradually adds Gaussian noise to the trajectory while a learned denoising process $p_\theta(\tau^{i-1}|\tau^i)$ iteratively refines the noise over N steps to recover trajectory τ^0.

To function as a planner, this sampling process is guided to satisfy task-specific objectives by sampling from a perturbed distribution:

$$\tilde{p}_\theta(\tau) \propto p_\theta(\tau)h(\tau) \tag{2.22}$$

Here, $p_\theta(\tau)$ ensures the plan is realistic and $h(\tau)$ acts as a guidance function that biases it towards the desired outcome. This decoupling of the base model from the goal is implemented in two ways:

1. Guided sampling is used for reward maximization, where the mean of the reverse process is shifted at each step by the gradient of the reward function $\nabla \mathcal{J}(\mu)$.

2. Inpainting is used for hard constraints, such as reaching a specific goal, where known values like the start and end states are fixed in the final denoised trajectory.

The use of generative sampling as planning allows for flexible behavior synthesis by composing the learned model with various guidance functions at test time. As a result, a single trained model can be adapted to diverse tasks without retraining, including:

- Offline reinforcement learning, where the model can be guided by different reward functions to perform competitively with specialized techniques.
- Goal-seeking policies for long-horizon and sparse-reward problems by fixing the start and end observations via inpainting. This was shown to be effective in complex maze environments.
- Arbitrary user-defined constraints, such as completing complex block-stacking configurations, by combining multiple perturbation functions.

2.5.2 Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

While Planning with diffusion treats planning as a conditional generation problem over entire state–action trajectories, Diffusion Policy focus on learning a reactive, visuomotor policy via imitation learning [2]. The paper addresses the core challenges of behavior cloning, where standard explicit policies (e.g., direct regression) struggle to model complex, multimodal action distributions, and implicit policies (e.g., energy-based models) suffer from training instability due to reliance on negative sampling. Chi et al. proposes to overcome these issues by representing the policy itself as a conditional denoising diffusion process.

This formulation can be understood as a practical, discretized implementation of the general reverse-time SDE introduced in equation 2.15. Instead of directly mapping an observation O_t to an action sequence A_t, the neural network $\epsilon_\theta(O_t, A_t^k, k)$ learns to predict the noise present in a noised action sequence A_t^k at diffusion step k.

At inference time, the policy starts with an action sequence A_t^K sampled from pure Gaussian noise and iteratively refines it over K steps to produce a clean, executable action sequence A_t^0. The denoising update at each step is given by:

$$A_t^{k-1} = \alpha \left(A_t^k - \gamma \epsilon_\theta(O_t, A_t^k, k) \right) + \mathcal{N}(0, \sigma^2 I). \tag{2.23}$$

The network is optimized with a simple mean-squared error loss between the true and predicted noise:

$$\mathcal{L} = \text{MSE}(\epsilon_k, \epsilon_\theta(O_t, A_t^0 + \epsilon_k, k)). \tag{2.24}$$

This formulation learns the score function of the conditional action distribution $p(A_t \mid O_t)$, which provides excellent distribution expressivity while remaining stable to train, unlike prior implicit methods.

To make this formulation practical for real-world robotics, Chi et al. combines the prediction of action sequences with receding horizon control, where the policy predicts a future sequence of actions (T_p) but only executes a shorter subsequence (T_α) before replanning. Second, it uses an efficient visual conditioning scheme where visual features are extracted once per inference cycle for real-time performance. Finally, the paper proposes Time-series diffusion transformer architecture to better handle tasks requiring high-frequency action changes. These contributions allow Diffusion Policy to consistently outperform prior state-of-the-art methods across numerous robot manipulation benchmarks.

2.6 Other Relevant Works

2.6.1 Q-Score Matching (QSM)

Q-Score Matching offers a novel training objective that reframes policy optimization from a geometric perspective[27]. The core insight is to directly align the score function of the diffusion policy, $\nabla \log \pi_\theta(a|s)$, with the action-gradient of the learned Q-function, $\nabla_a Q(s, a)$. In other words, the policy is trained such that its vector field of log-probabilities is iteratively "pushed" to match the direction of steepest ascent in the Q-value landscape.

This approach provides a direct, analytical objective for policy improvement that completely avoids the need to differentiate through the diffusion sampling chain. By exploiting the inherent score-based structure of the diffusion model, QSM enables the stable and

sample-efficient learning of complex, multi-modal policies that can more effectively cover the modes of the optimal action distribution.

2.6.2 Diffusion Steering via Reinforcement Learning (DSRL)

In contrast to modifying the training objective, Diffusion Steering via Reinforcement Learning (DSRL) is an inference-time optimization technique [28]. It leaves the pre-trained diffusion policy fixed and instead focuses on the initial latent noise, $wN(0, I)$, which is the seed for the denoising process. The key idea is to train a secondary, lightweight "steering" policy that learns to select an initial noise vector w that, when denoised by the fixed diffusion model, produces a high-value action.

This method also tries to address counterfactual query: "What if a different initial noise had been chosen?" By learning to navigate the latent noise space, DSRL can rapidly steer the powerful generative prior towards actions that maximize rewards for a specific task. This allows for extremely fast and sample-efficient policy adaptation without the computational cost or potential instability of fine-tuning the large, pre-trained diffusion model itself.

2.6.3 Flow Q-Learning

A third, highly effective approach to the policy extraction problem is presented in Flow Q-Learning (FQL) [29]. This method elegantly circumvents the challenges of direct policy optimization by decoupling the learning of the expressive behavioral prior from the value maximization task itself. FQL uses a "teacher-student" distillation framework [30].

First, an iterative, flow-matching policy is trained solely with behavioral cloning (BC) to serve as an expressive "teacher" model that perfectly captures the complex, multimodal action distributions in the expert dataset. The reinforcement learning objective is then offloaded to a separate, simple one-step "student" policy. This student policy is trained to maximize the learned Q-function, but it is simultaneously regularized by a distillation loss that forces its output to stay close to that of the powerful flow-matching teacher. This strategy is extremely clever because it completely avoids the unstable and computationally expensive backpropagation through the iterative ODE solver, and the final deployed policy is the efficient one-step student. Costly iterative sampling at inference time is therefore completely avoided. FQL thus achieves the best of both worlds, a distributional

expressivity of a flow model with the training stability and inference speed of a simple policy.

3 METHODOLOGY

This chapter details the methodology for the proposed Flow Planner framework, a novel hierarchical framework for long-horizon trajectory planning. This chapter is structured to build from foundational theory to the specific contributions of this work.

Section 3.1 first introduces the theoretical underpinnings of Conditional Flow Matching (CFM), a powerful, recently proposed paradigm for generative modeling. Building upon this, Section 3.2 introduces a clever reformulation of the CFM objective, specializing this framework to the complex domains of robotic trajectory planning. This section goes over the general formulations for each planner component developed in this dissertation. Finally, Section 3.3 describes how these components are integrated into the complete hierarchical framework.

The formulations presented herein are adapted from the convention from "An Introduction to Flow Matching and Diffusion Models" by MIT CSAIL [6], as well as the "Flow Matching Guide and Code" provided by Meta [7].

3.1 Conditional Flow Matching Formulation

Recall in Section 2.4.3, the general Flow Matching objective 2.20 is intractable because it requires access to the marginal probability path $p_t(x)$ and its corresponding vector field $u_t(x)$. Conditional Flow Matching (CFM) is a practical implementation that solves this problem by defining the probability path such that it yields a tractable target vector field. This transforms the objective into a stable and efficient regression problem.

3.1.1 CFM Training

CFM constructs the path as a simple linear interpolation between a point sampled from a prior distribution $x_0 \sim p_0$ and a point from the data distribution $x_1 \sim p_1$. The prior is usually standard Gaussian noise $p_0(x) = \mathcal{N}(0, I)$. The point on the path at any time $t \in [0, 1]$ is defined as:

$$x_t = (1 - t)x_0 + tx_1. \tag{3.1}$$

This specific construction is a type of Gaussian probability path referred to as the CondOT (Conditional Optimal Transport) path [7]. The ground truth vector field requiring to move a particle along this straight path is constant at all times, and it is simply the vector connecting the two endpoints:

$$u_t(x_t) = x_1 - x_0. \tag{3.2}$$

This replaces the intractable integral of the general formulation with a simple difference of two vectors, which are readily available during training. By substituting this simplified target vector field into the general objective in 2.20, one arrives at the final CFM loss function. The goal is to train a neural network $v_\theta(x_t, t, c)$ to predict the target vector $(x_1 - x_0)$ given the interpolated point x_t and time t, and any optional conditioning variables c. This objective is a straightforward L2 regression loss:

$$L_{CFM}(\theta) = E_{t,p_0(x_0),p_1(x_1)}[\|v_\theta((1-t)x_0 + tx_1, t, c) - (x_1 - x_0)\|^2]. \tag{3.3}$$

A single training step is summarized in Algorithm 1

Algorithm 1 CFM Training

Require: Dataset of samples $\mathbf{x}_1 \sim p_{\text{data}}$, neural network v_θ, prior distribution p_0 (e.g. $\mathcal{N}(0, I)$)

for each training step **do**
  $\mathbf{x}_1 \sim p_{\text{data}}$ ▷ Sample a data point
  $\mathbf{x}_0 \sim p_0$ ▷ Sample a noise vector
  $t \sim \text{Unif}[0, 1]$ ▷ Sample a random time
  $\mathbf{x}_t \leftarrow (1-t)\mathbf{x}_0 + t\mathbf{x}_1$ ▷ Compute interpolated point
  $\mathbf{u} \leftarrow \mathbf{x}_1 - \mathbf{x}_0$ ▷ Compute target vector
  $\mathcal{L}(\theta) \leftarrow \|v_\theta(\mathbf{x}_t, t, c) - \mathbf{u}\|^2$ ▷ Compute the loss
  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$ ▷ Gradient descent update
end for

This is a simulation-free objective, hence it only requires sampling one data point, one noise vector, and a time to perform a standard regression update. It avoids the costly ODE solves of traditional CNF training and forms the foundational recipe for the specific planners detailed in the following sections. While the CondOT path is a common choice, the implementation in this dissertation utilizes a more general affine probability path. This formulation, known as the "Affine Conditional Flow Matching" variant in Albergo et al. [cite here], defines an exact conditional distribution over the interpolants instead of a single deterministic interpolation. This preserves key probabilistic properties, leading

to more stable training dynamics and a closer theoretical alignment with Schrödinger Bridge formulations [31]. The detailed derivation of the base CFM objective is provided in Appendix A.2.

3.1.2 CFM Inference

While the training process for CFM is simulation-free, inference requires simulating the learned dynamics. The goal of inference is to generate a new sample x_1 that approximates a draw from the true distribution by solving an Ordinary Differential Equation (ODE). The trained neural network $v_\theta(x_t, t, c)$ now serves as an approximation of the true (but intractable) marginal vector field that transforms noise into data. Generation begins by sampling an initial point from the prior distribution $\mathbf{x}_0 \sim p_0(\mathbf{x})$. This point is then evolved from $t = 0$ to $t = 1$ by solving the following ODE:

$$\frac{d\mathbf{x}t}{dt} \;=\; v_\theta(\mathbf{x}_t, t, c). \tag{3.4}$$

The solution to this ODE, x_1, is the final generated sample. Since this ODE rarely has an analytical solution, it must be solved numerically. A common choice is the simple **Euler method**, detailed in Algorithm 2.

Algorithm 2 Euler Method

Require: Neural network v_θ, prior distribution p_0, condition c, number of steps N

$\mathbf{x}_0 \sim p_0(\mathbf{x})$ ▷ Sample an initial point
$\Delta t \leftarrow 1/N$
for $i = 0$ to $N - 1$ **do**
    $t \leftarrow i \cdot \Delta t$
    $\mathbf{x}_{t+\Delta t} \leftarrow \mathbf{x}_t + v_\theta(\mathbf{x}_t, t, c) \cdot \Delta t$ ▷ Take a step along vector field
end for
return $\mathbf{x}_1$

It is worth noting that for Stochastic Differential Equations (SDEs), the corresponding simple solver is the Euler-Maruyama method. For the experiments in this dissertation, a more advanced solver, **the midpoint method** (a 2nd-order Runge-Kutta scheme) is employed [7]. This choice offers a better trade-off between computational cost and numerical stability, leading to higher-quality samples compared to the basic Euler method.

3.2 Planners Formulation

The general Conditional Flow Matching (CFM) objective from Section 3.1 provides a simulation-free training recipe. While this framework has been successfully specialized for a range of generative modeling tasks, particularly in domains such as image and video synthesis, its application to trajectory planning remains largely unexplored.

This section details the novel specialization of the CFM framework for trajectory planning. To present the methodology in a unified manner, this section first presents a generalized planner formulation and then details how this single framework is instantiated for three different planners: Joint Planner (JP), Waypoint Planner (WP), and Action Planner (AP). Note the distinction between the overall task horizon T, which can be arbitrarily long, and the fixed planning horizon of the model H, which is a number of steps the generative model is trained to predict. This distinction is key to how the framework manages long-horizon problems.

3.2.1 Generalized Planner Formulation

Let τ represent a generic trajectory data sample. This sample can be a state-action sequence, a state-only sequence, or an action-only sequence over the planning horizon H and let c be a set of conditioning variables. The goal is to learn a conditional distribution $p(\tau \mid c)$ by training a neural network $v_\theta(\tau_t, t, c)$. The generalized CFM objective for any planner is given by:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\tau,\tau_{noise}} \| v_\theta(\tau_t, t, c) - (\tau - \tau_{noise}) \|, \tag{3.5}$$

where τ is a ground-truth trajectory from the dataset, $\tau_{noise} \sim \mathcal{N}(0, I)$, and

$$\tau_t = (1 - t)\tau_{\text{noise}} + t\tau. \tag{3.6}$$

3.2.2 Planner Specializations

The three planners implemented below are specific instances of the above generalized formulation, differing only in their choice of trajectory data τ and conditioning variables c.

3.2.2.1 Joint State-Action Planner (JP)

The Joint Planner is designed to generate a complete, long-horizon strategy that includes both states and actions. It learns the conditional distribution over entire state-action trajectories $p(\tau \mid c)$, where $c = (s_{start}, s_{goal})$. A JP outputs a trajectory designed over the planning horizon H:

$$\tau = (s_0, a_0, ..., s_H, a_H). \tag{3.7}$$

This formulation is inspired by the inpainting-based planning approach of Janner et al, who demonstrated the feasibility of generating trajectories within a diffusion model framework [4]. However, whereas their method relied on a separate guidance function $\mathcal{J}_\phi$ to steer the sampling process, my CFM-based implementation models the conditional distribution directly. While this end-to-end approach is theoretically powerful (and intuitive) for open-loop planning, modeling the full joint state-action space over a long horizon is often practically infeasible. The subsequent planners are designed to mitigate this by decoupling the problem.

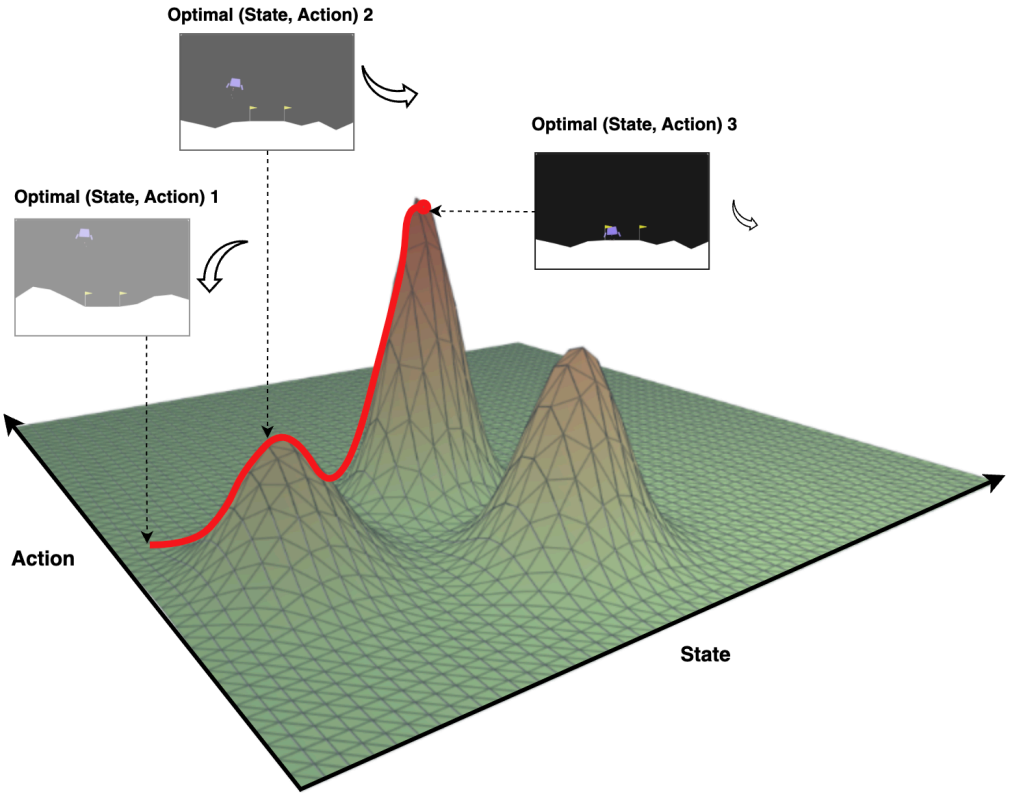

Figure 3.1: Joint State-Action Planner Visualized

3.2.2.2 State-Only Waypoint Planner (WP)

I proceed to decouple the high-level pathfinding from the low-level control. The Waypoint Planner generates a long-horizon strategic plan consisting only of states, which serve as a sequence of waypoints for a low-level controller, also defined over the planning horizon H:

$$\tau_s = (s_0, ..s_H). \tag{3.8}$$

The Waypoint Planner learns the conditional distribution over state-only trajectories, $p(\tau_s \mid c)$, where the conditioning $c = (s_{start}, s_{goal})$ remains the same.

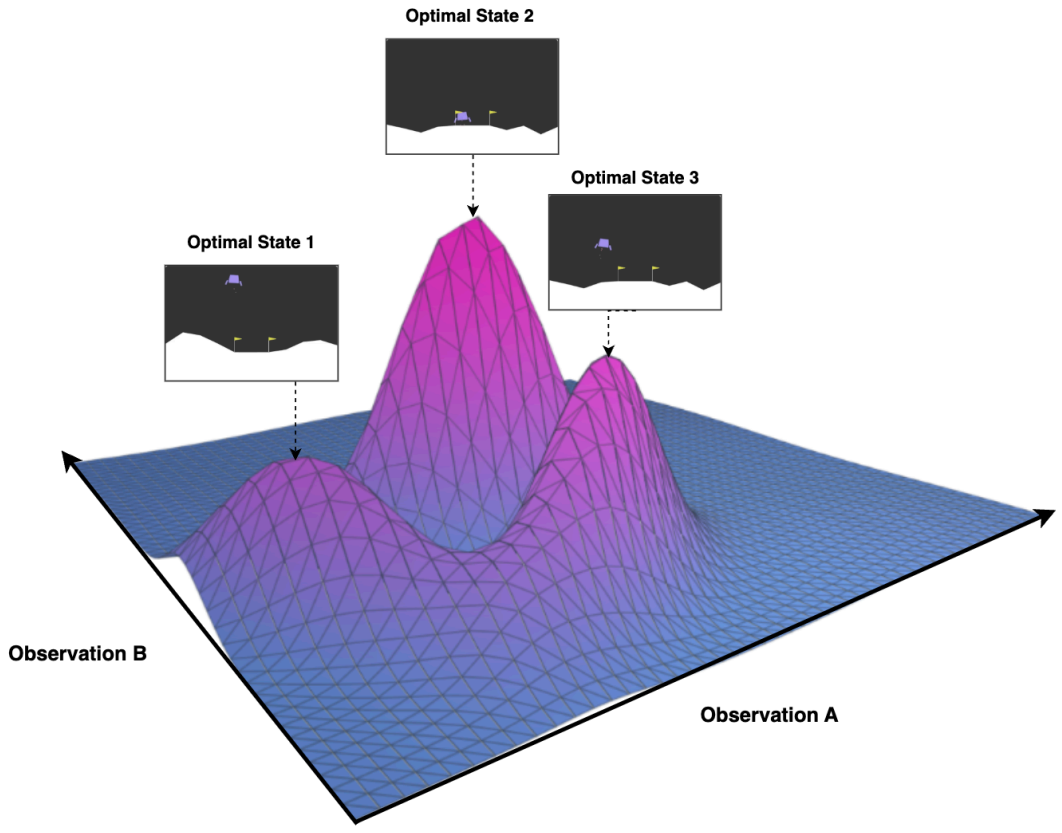

Figure 3.2: State-Only Waypoint Planner Visualized

3.2.2.3 Action Planner (AP)

Inspired by the work of Chi et al.'s on Diffusion policy, the Action Planner is designed to be a reactive, low-level policy [2]. Its role is to generate a short-horizon sequence of actions

that guides the agent from its current state to a nearby waypoint, typically provided by a high-level planner like the WP.

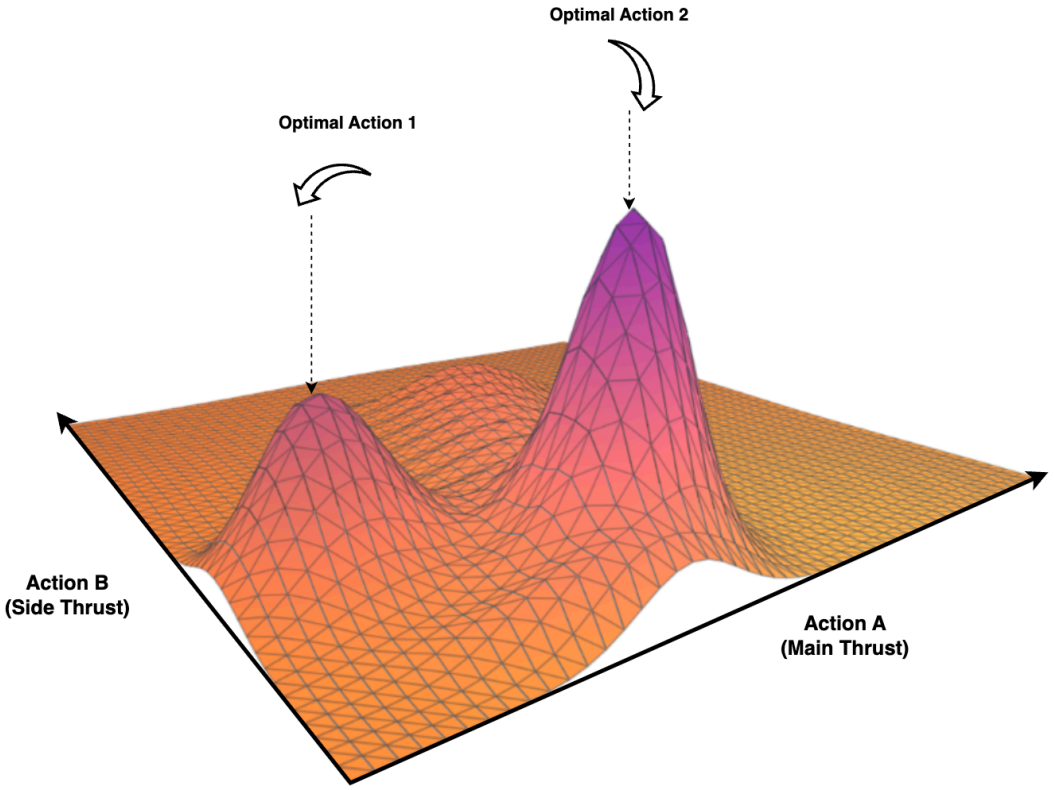

Figure 3.3: Action Planner Visualized

It learns the conditional action distribution $p(\tau_a \mid c)$, where the conditioning is $c = (s_{current}, s_{waypoint})$. Note the action sequence is defined over a short action horizon H_a , where $H_a \ll H$.

$$\tau_a = (a_0, \ldots, a_{H_a}). \tag{3.9}$$

A summary of each planner can be found in Table 3.1

Table 3.1: Overview of planners, their models, trajectory data, and conditioning variables.

Planner	**Model**	**Trajectory Data (τ)**	**Conditioning Variables (c)**
Joint Planner	v_θ^{JP}	$(s_0, a_0, \ldots, s_H, a_H)$	$(s_{\text{start}}, s_{\text{goal}})$
Waypoint Planner	v_θ^{WP}	$(s_0, \ldots, s_H)$	$(s_{\text{start}}, s_{\text{goal}})$
Action Planner	v_θ^{AP}	$(a_0, \ldots, a_{T_a})$	$(s_{\text{current}}, s_{\text{waypoint}})$

3.3 Flow Planner Framework

The individual planners in Section 3.2 are composed into a hierarchical framework that tackles long-horizon tasks by decoupling high-level strategic reasoning from low-level reactive control. This section details the framework's operational flow and motivates the key design choices. The architecture pairs a state-only Waypoint Planner (WP) at the high level with an Action Planner (AP) at the low level, executed within a Model Predictive Control (MPC) loop. MPC follows a receding-horizon scheme: at each step a finite-horizon open-loop plan is computed; only its first action is applied, after which the horizon is shifted and the optimization re-solved [32].

3.3.1 Hierarchical Planning Process

The planning process unfolds in two distinct stages: an initial strategic planning phase and a continuous, reactive control loop.

1. **Global Plan Generation**: At the beginning of a planning cycle, the Waypoint Planner (WP) is invoked to generate a strategic plan, $\tau_s = (s_0, \ldots, s_H)$, over its fixed planning horizon H. The conditioning signal for this planner, $c = (s_{start}, s_{goal})$ can be formulated in two ways, each with distinct purpose:

   - **Final Goal Conditioning**: The planner can be conditioned on the final goal of the entire task horizon, where $s_{goal} = s_T$. This provides the model with the ultimate objective, encouraging it to generate a plan that is globally directed towards the final destination.
   - **Waypoint Goal Conditioning**: Alternatively, the planner can be conditioned on an intermediate goal at the end of its planning window, $s_{goal} = s_H$. In this receding-horizon approach, the planner solves a shorter, more immediate

problem. For tasks whose total duration exceeds the planning horizon ($T \gg H$), the WP can be invoked in a low-frequency receding-horizon manner as well to generate the next segment of the plan once the current one is near completion.

2. **Reactive MPC Loop**: The system then enters a high-frequency MPC loop to execute the plan:

   (a) The agent's current state, $s_{current}$, is observed.

   (b) The next immediate waypoint, $s_{waypoint}$, is selected from the global plan τ_s.

   (c) The AP is conditioned on both the current state and the target waypoint, $c = (s_{current}, s_{waypoint})$ and generates a short-horizon action sequence, $\tau_a = (a_0, \ldots, a_{T_a})$.

   (d) The first action a_0 from this sequence is executed in the environment.

   (e) The process repeats, with the agent progressing through the waypoints of the global plan.

The full procedure is detailed in Algorithm 3.

Algorithm 3 Hierarchical Planning

Require: Trained Waypoint Planner v_θ^{WP}, Trained Action Planner v_θ^{AP}, initial state $\mathbf{s}_{\text{start}}$, goal state $\mathbf{s}_{\text{goal}}$
Ensure: Action sequence leading to $\mathbf{s}_{\text{goal}}$

$\tau_s \leftarrow \text{sample}\Big(v_\theta^{WP} \mid \mathbf{s}_{\text{start}}, \mathbf{s}_{\text{goal}}\Big)$ ▷ Generate high-level plan
$i \leftarrow 1$
$\mathbf{s}_{\text{current}} \leftarrow \mathbf{s}_{\text{start}}$
while not terminated **do**
    $\mathbf{s}_{\text{waypoint}} \leftarrow \tau_s[i]$
    $\mathbf{A}_{\text{seq}} \leftarrow \text{sample}\Big(v_\theta^{AP} \mid \mathbf{s}_{\text{current}}, \mathbf{s}_{\text{waypoint}}\Big)$
    $a_0 \leftarrow \mathbf{A}_{\text{seq}}[0]$
    $\mathbf{s}_{\text{next}} \leftarrow \text{Execute}(a_0)$ ▷ Execute action and observe next state
    $\mathbf{s}_{\text{current}} \leftarrow \mathbf{s}_{\text{next}}$
    if $\text{distance}(\mathbf{s}_{\text{current}}, \mathbf{s}_{\text{waypoint}}) < \text{threshold}$ **then** ▷ Check if waypoint was reached
        $i \leftarrow i + 1$
    end if
end while

3.3.2 Framework Design Philosophy

The design of the Flow Planner framework is motivated by the inherent limitations of reactive policies. While powerful for short-horizon tasks, the Diffusion Policy framework

proposed by Chi et al can exhibit a "gold fish effect" where a limited observation history makes it difficult to maintain coherence over long durations [4]. This is particularly challenging in tasks with long-horizon multimodality, where the policy may lack sufficient history to disambiguate between different strategic modes, leading to indecisive or "stuck" behaviors.

On the other hand, end-to-end approaches that excel in open-loop planning like the inpainting-based planner proposed by Janner et al. runs into a different scalability issue [2]. While theoretically capable of generating long plans, modeling the entire joint state-action space over a long horizon is often practically infeasible due to the sample complexity and computational cost associated with such high-dimensional model inputs.

Therefore, the Flow Planner Framework is centered around a "hierarchical decomposition" philosophy designed to address both of these challenges simultaneously. It is also worth noting that it is grounded in probabilistic factorization. Using the chain rule of probability, one can factorize the original complex joint state-action space distribution into two simpler conditional parts:

$$p(\tau_s, \tau_a \mid c) = \underbrace{p(\tau_s \mid c)}_{\text{Waypoint Planner}} \cdot \underbrace{p(\tau_a \mid \tau_s, c)}_{\text{Action Planner}}. \tag{3.10}$$

While this dissertation focuses on a separate training scheme to maximize modularity, this factorization also provides a theoretical foundation for a joint training paradigm. The new objective and its implications for learning a disentangled representation are detailed in Appendix A.3. By decoupling long-horizon planning into strategic reasoning and reactive control, this framework gains several advantages:

1. **Computational Efficiency**: The most computationally intensive component, the long-horizon Waypoint Planner, is invoked infrequently. For tasks shorter than its maximum horizon H, it is called only once. For extremely long-horizon tasks, it can be re-invoked periodically to generate the next strategic plan. In contrast, the lightweight Action Planner operates within the high-frequency MPC loop. The overall planning cost is therefore amortized.

2. **Reduced Learning Complexity**: The learning task for each component is dramatically simplified. The waypoint planner is freed from learning fine-grained actions and can focus solely on the high-level structure of the state space. Conversely, the action planner only needs to solve a simple, short-horizon goal-reaching problem.

3. **Modularity and Separation of Concerns**: Modularity and Separation of Concerns: The framework's modular design allows for the independent development, optimization, and debugging of the high-level and low-level planners. The decoupling of perception from planning also allows seamless adaptation to different observation spaces. For high-dimensional, vision-based environments, a VAE encoder [17] can be inserted to handle the perception task, allowing the planners to operate in a compressed latent space, as shown in the Figure 3.4 and 3.5 [33]. This separation of concerns also facilitates the integration of hybrid systems where, for example, a learned generative high-level planner could be paired with a classical low-level controller like an LQR or PID.

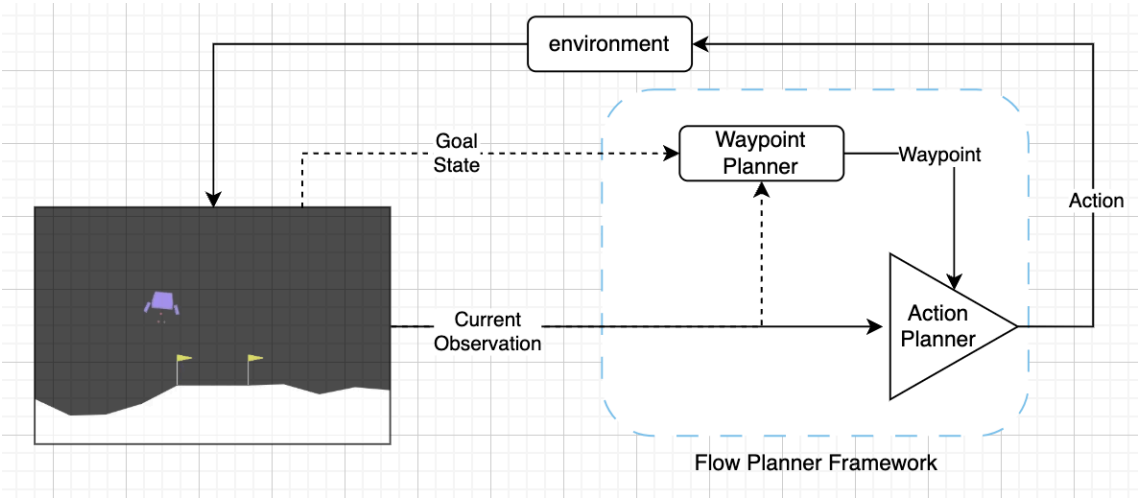

Figure 3.4: Flow Planner Framework for Agent-centric Observations

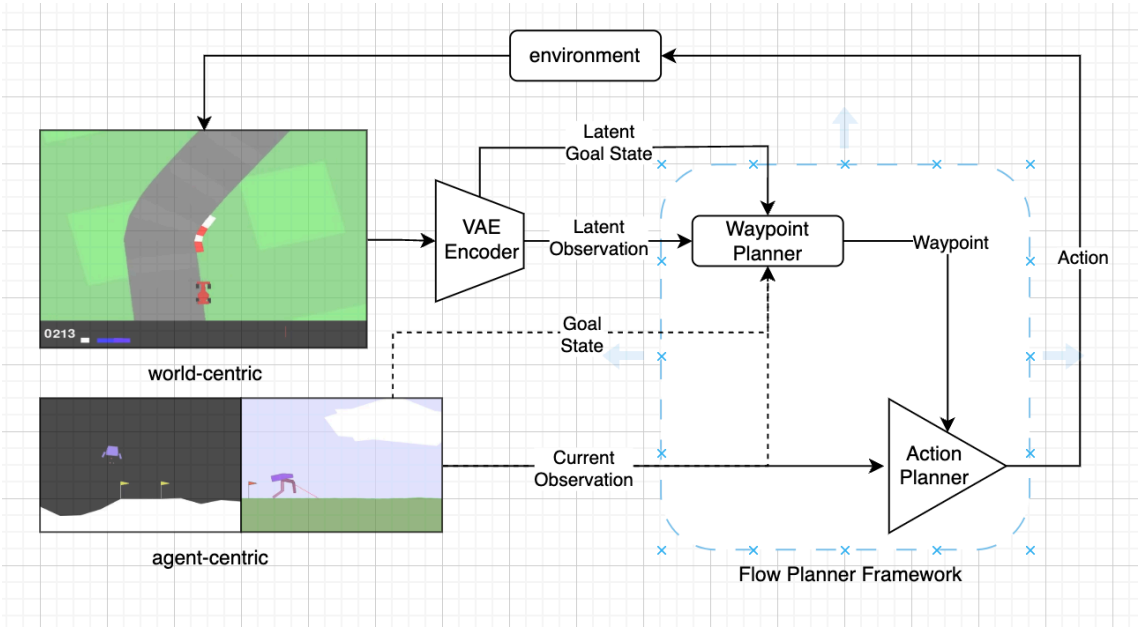

Figure 3.5: Flow Planner Framework Adapted for World-centric Observations

4 DATASET PREPARATION

This chapter details the data pipeline designed to generate the expert trajectories used to train and evaluate the Flow Planner framework. The process begins with the selection of a suitable evaluation environment, followed by the training of an expert agent using Proximal Policy Optimization (PPO) to generate the demonstration dataset. The datasets as well as the trained agents can be found on my personal Hugging Face Hub.

4.1 Environment Suite

To test the capabilities of a generative planner, a suite of three distinct environments from the Gymnasium [8] Library was selected, each representing a unique set of challenges. While this dissertation will focus on the empirical evaluation of LunarLanderContinuous-v3 for a more focused analysis, the other environments are included here for future research into scaling the Flow Planner framework to more complex locomotion and vision-based tasks.

4.1.1 LunarLanderContinuous-v3

This environment presents a classic control problem where the agent's objective is to land a craft safely and with minimal fuel consumption between two designated flags. The low-dimensional state and continuous dynamics makes it an ideal benchmark for testing an agent's ability to learn precise thrust control and solve a delayed credit assignment problem, where early actions have critical consequences. A successful landing requires scoring over 200 points, which incorporates rewards for proximity, velocity, stability, and fuel efficiency, with a bonus of +100 for a safe landing and -100 for crashing.

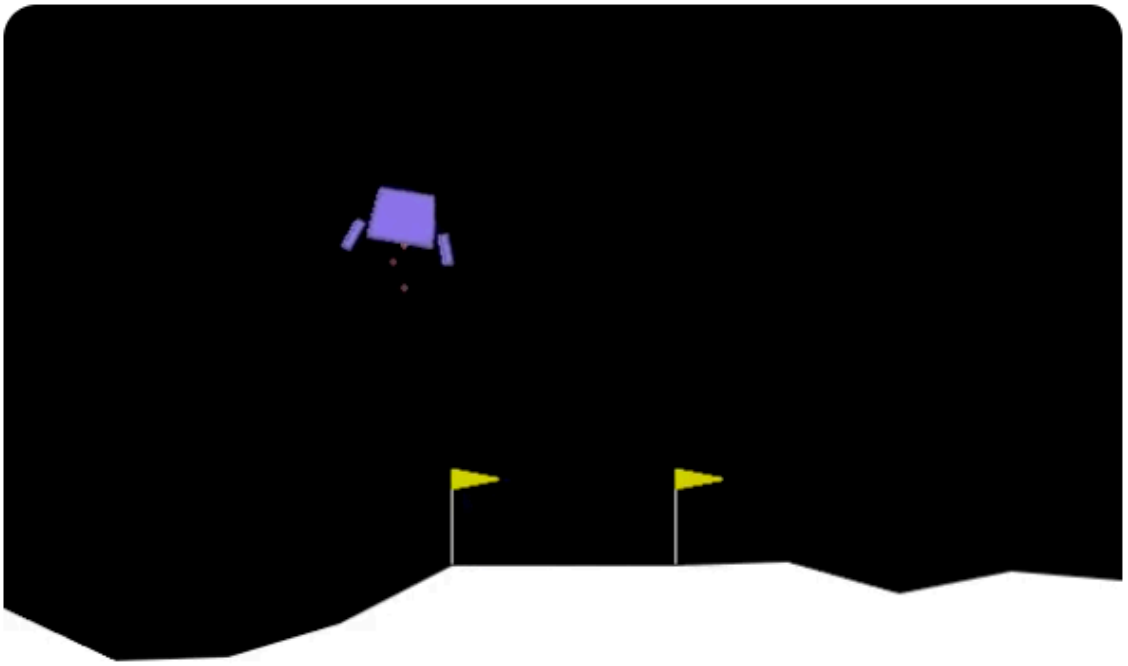

Figure 4.1: LunarLanderContinuous-v3 Environment

4.1.2 BipedalWalker-v3

This environment increases the complexity by focusing on continuous locomotion over a randomly generated, uneven terrain. Success requires the agent to master the coordination of multiple joints to develop a robust and efficient walking gait, making it a challenging benchmark for learning stable locomotion policies.

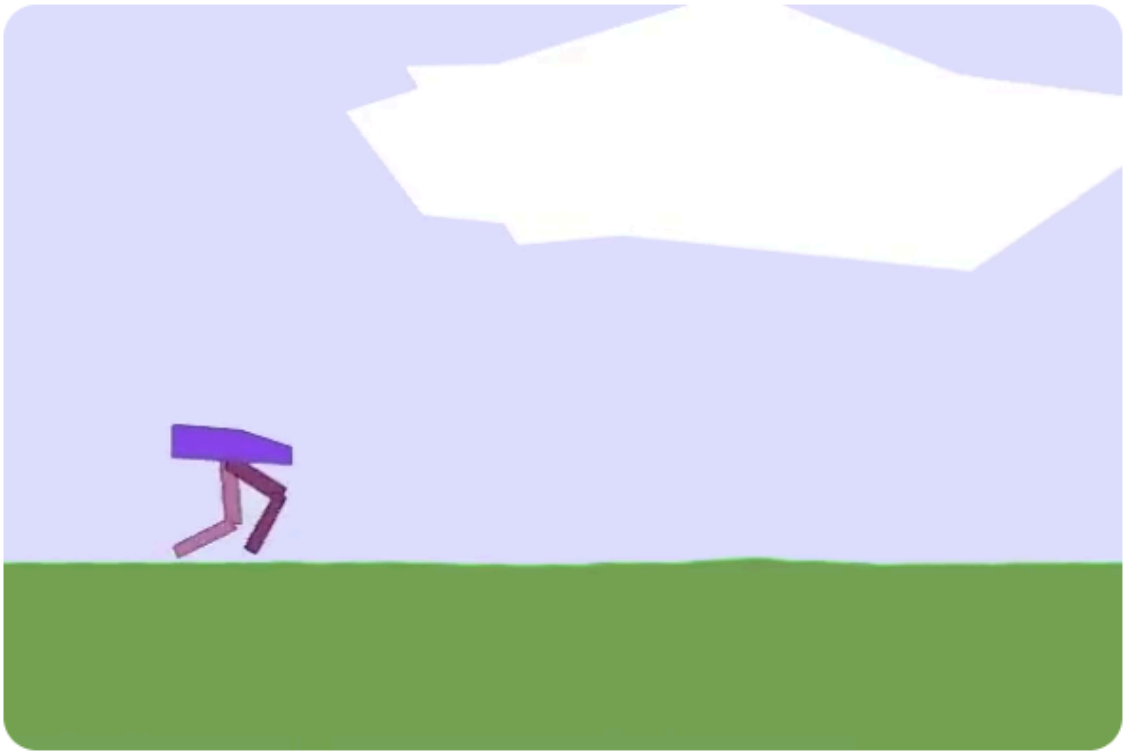

Figure 4.2: BipedalWalker-v3 Environment

4.1.3 CarRacing-v3

This environment introduces the challenge of vision-based control from high-dimensional 96x96 pixel images. The objective is to complete a lap on a procedurally generated track, testing the agent's representation learning capabilities and its ability to perform high-speed continuous control from raw pixels. The detailed procedure for processing these visual observations and generating the final latent dataset is provided in Appendix A.4.

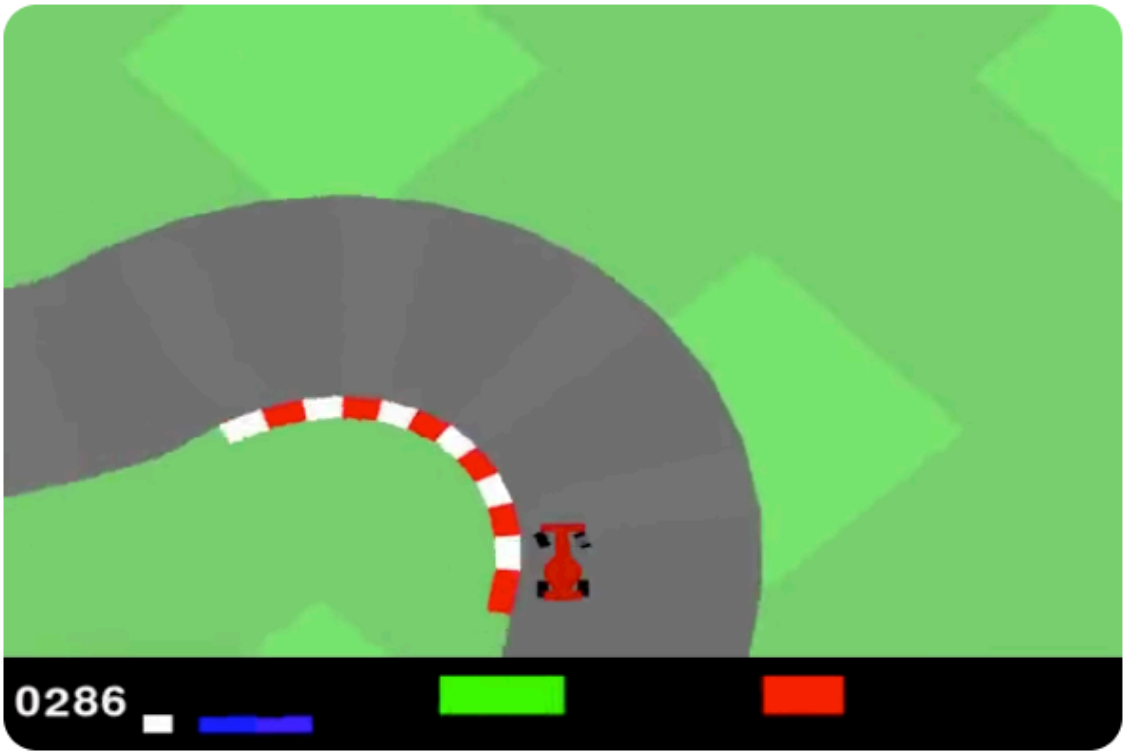

Figure 4.3: CarRacing-v3 Environment

4.2 Expert Policy Training

To generate the expert datasets, a Proximal Policy Optimization (PPO) agent was trained in each environment until convergence. PPO was chosen for its exceptional training stability. As a state-of-the-art on-policy algorithm, it optimizes the policy by taking a series of small, conservative steps. It uses a "clipped" surrogate objective that acts as a guardrail, preventing any single update from changing the policy too dramatically [34]. This inherent stability makes PPO a robust workhorse for the continuous control challenges in these environments.

All agents were trained using the tuned hyper-parameters supplied by the RL Baselines3 Zoo [35] library, with the final performance summarized in Table 4.1, 4.2 and 4.3.

Table 4.1: PPO expert (LunarLanderContinuous-v3).

Mean Reward	236.78 ± 79.67
Wrappers	None
Observation Space	Box(8,) float32
Action Space	Box(2,) float32

Table 4.2: PPO expert (BipedalWalker-v3).

Mean Reward	271.87 ± 2.08
Wrappers	None
Observation Space	Box(24,) float32
Action Space	Box(4,) float32

Table 4.3: PPO expert (CarRacing-v3).

Mean Reward	248.24 ± 168.67
Wrappers	**FrameSkip, Resize, GrayScale, FrameStack**
Observation Space	Box(64, 64, 2) uint8
Action Space	Box(3,) float32

To ensure a comprehensive set of expert policies, agents were also trained using the A2C and TD3 algorithms. Although these methods demonstrated superior performance in certain environments, solely the PPO agents were used to generate the expert datasets. This decision was made to prioritize the quality and stability of the expert behavior for imitation. PPO's on-policy nature and clipped objective function consistently produce smooth and generalizable policies, which serve as a more robust and reliable foundation for training a generative model. While off-policy methods like TD3 can be highly performant, they can also yield more exploitative or brittle policies that are less suitable as a behavioral prior. For the benefit of future research and to ensure reproducibility, all trained models, including the A2C and TD3 agents, have been made publicly available on my Hugging Face Hub.

4.3 Expert Dataset Collection

To train the generative planners described in Chapter 3, a dataset of expert trajectories were required for each environment. This data was collected by executing the trained policies from Section 4.2. The Minari [9] Library's DataCollector wrapper was used to systematically record trajectory data during these rollouts. Approximately 300,000 to 350,000 steps were sampled for each environment to create a robust dataset capturing a wide range of expert behaviors. The resulting datasets are summarized in the table below. Note the number of episodes collected for each environment was tailored to its specific characteristics. For instance, the stochastic nature of LunarLander-v3 necessitated a large number of episodes (1000) to ensure a diverse set of successful landing trajectories. On the other hand, the high-dimensional visual observations in CarRacing-v3 resulted in a substantial dataset size from fewer but longer episodes (250). The dataset statistics for different environments are summarized in Table 4.4.

Table 4.4: Dataset statistics for different environments.

Environment	**LunarLanderContinuous-v3**	**BipedalWalker-v3**	**CarRacing-v3**
# Episodes	**1000**	500	250
# Steps	335K	**346K**	245K
Size	33.7 MB	49.9 MB	**446.8 MB**

Again, to ensure full reproducibility and to facilitate future research, these datasets have been version-controlled and are publicly available on my personal Hugging Face Hub.

5 EXPERIMENTAL RESULTS

This chapter presents the empirical evaluation of the proposed Flow Planner Framework, with experiments designed to validate the performance of the hierarchical planning methodology and to analyze the impact of different architectural choices and conditioning signals. To ensure fair and reproducible comparisons, all experiments were conducted on a workstation equipped with an NVIDIA RTX 4080 Super GPU and an AMD Ryzen 7 7800X3D CPU. The LunarLander-v3 environment serves as the primary testbed for this analysis.

This chapter is structured as follows: Section 5.1 details and compares the performance of different neural network architectures for both flow matching and diffusion-based planners [4]. Leveraging the best-performing architecture from this initial analysis, Section 5.2 establishes a performance baseline by evaluating the monolithic Joint Planner under two different conditioning schemes. Finally, Section 5.3 evaluates the core contribution of this work: the decoupled, hierarchical planner that combines the Waypoint and Action Planners. Each of these experimental sections will present both qualitative analysis of the open-loop trajectory generation and a qualitative, closed-loop MPC evaluation to measure the final planner's performance via cumulative rewards over 10 episodes.

Unless otherwise specified, all models for the remainder of Section 5 were trained for 100 epochs with a learning rate of 1e-3. For CNN-based models, the kernel size was standardized to 5. For the diffusion model comparison, models were trained with 1000 diffusion steps and sampled using 100 inference steps with a standard DDPM scheduler.

5.1 Neural Network Architecture Comparisons

The initial phase of this empirical study is to determine the most effective architectural backbone. A generative planner's performance is directly tied to its network architecture, especially for sequential data such as a trajectory that possesses a strong temporal structure. This section establishes a simple MLP as a baseline, then evaluates its performance against two more sophisticated architectures: a 1D CNN [36] and a U-Net [37]. Each architecture is implemented for both our novel flow matching planners and the existing

diffusion-based models (used as baselines), to determine the optimal neural network backbone for the planning experiments that follow. Note that all models in this section are trained on the unconditional tasks of modeling the full state-action trajectories.

5.1.1 MLP Baseline

As a foundational baseline, a simple Multi-Layer Perceptron (MLP) is used. The network first processes a flattened representation of the entire trajectory and projects it into a hidden dimension. This is then combined with a sinusoidal time embedding before being passed through a series of feedforward layers [38]. While computationally efficient, this architecture deliberately ignores the inherent temporal coherence of the data. Nevertheless, it serves as a crucial baseline to quantify the performance gains achieved by more structured, sequence-aware architectures. The detailed implementation is provided below.

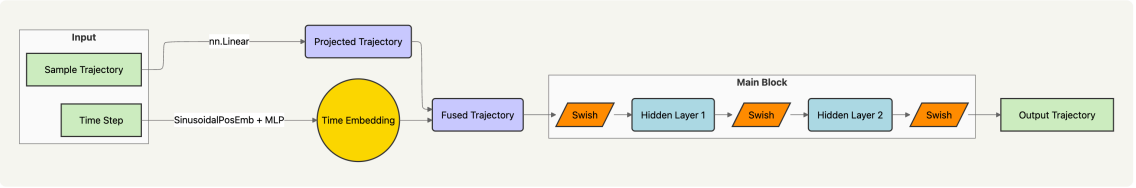

Figure 5.1: MLP Implementation

The MLP baseline is a fully-connected neural network designed to process the entire trajectory as a single, flattened vector. The architecture consists of three main components: an initial projection, a time embedding module, and a main processing block.

1. **Input Processing:** The input trajectory x, with shape (batch_size, horizon, features), is first flattened. This flattened vector is then passed through an nn.Linear layer (initial_projection) to project it into a hidden_dim vector space.

2. **Time Embedding:** The time step t is processed by a dedicated time embedding module. It is first converted into a high-dimensional representation using a SinusoidalPosEmb layer, which is then passed through a two-layer MLP (Linear → Swish → Linear) to produce a final time embedding vector, also of size hidden_dim.

3. **Core Computation:** The projected input vector and the time embedding vector are combined via element-wise addition to form the main hidden state h.

4. **Main Block:** This hidden state is then processed by a three-layer MLP, where each nn.Linear layer is followed by a Swish activation function. The final layer projects the representation back to the original flattened input dimension. The output is then reshaped to match the original trajectory's shape.

5.1.2 1D CNN

To explicitly model the temporal dependencies within the trajectory data, a 1D Temporal Convolutional Neural Network is implemented. This architecture, inspired by its successful application in the Diffuser framework, reshapes the trajectory into channels by sequence length format. A series of 1D convolutional layers then operate along the time axis, allowing the model to effectively capture local temporal patterns and dependencies between adjacent state-action pairs. A detailed implementation is provided below.

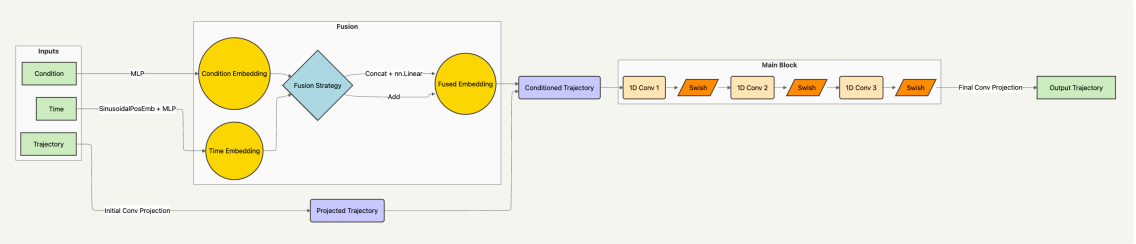

Figure 5.2: CNN Implementation

The 1D Temporal CNN is designed to process trajectories as sequences, explicitly modeling temporal dependencies. Its architecture is composed of projection layers, embedding modules with a flexible fusion strategy, and a main convolutional block.

1. **Input Processing:** The input trajectory x is reshaped from a flattened vector into a (batch, transition_dim, horizon) sequence. A 1x1 convolution (initial_conv) is then used to project the feature dimension (transition_dim) into the hidden_dim.

2. **Embedding and Fusion:** The model processes the time t and conditioning variable c through separate two-layer MLP embedding networks. The resulting embeddings are combined into a single vector using a specified fusion_strategy. Depending on the strategy, the embeddings are either added element-wise or concatenated. The fused embeddings then pass through an additional linear layer to project them back to hidden_dim.

3. **Core Computation:** The fused embedding vector is broadcasted and added to the hidden state h produced by the initial convolution.

4. **Main Block:** The core of the network is a sequence of three Conv1d layers with a kernel size of 5 and "same" padding, with Swish activations between them. This block processes the trajectory along its temporal axis.

5. **Output:** A final 1x1 convolution (final_conv) projects the hidden representation back to the original transition_dim (feature dimension of each timestep, e.g. state-action). The output is then reshaped to match the original flattened trajectory format.

5.1.3 U-Net

To further enhance the model's ability to capture features at multiple timescales, a U-Net architecture is adapted. Originally designed for medical imaging [cite here] , the standard 2D convolutions are replaced with 1D temporal convolutions. This allows the model's encoder-decoder structure and skip connections to capture both coarse, long-range dependencies and fine-grained, local patterns. This multi-scale processing makes the U-Net a highly expressive architecture for modeling complex, long-horizon trajectories. Again, a detailed implementation is provided below.

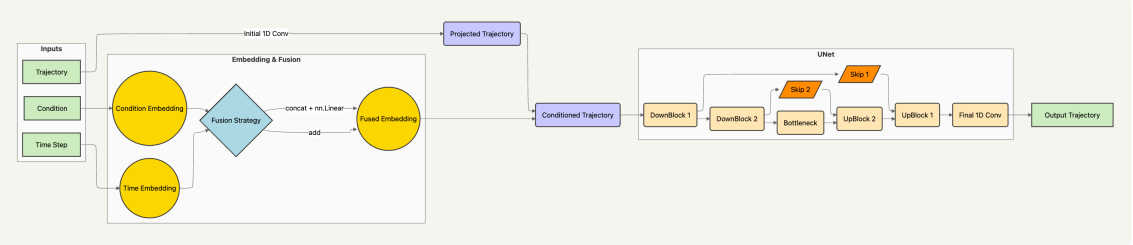

Figure 5.3: U-Net Implementation

The 1D Temporal U-Net is a more sophisticated convolutional architecture that processes sequential data at multiple resolutions to capture both local and global temporal patterns. It is built upon the same input processing and embedding fusion logic as the 1D CNN.

1. **Input and Embedding:** The initial processing of the trajectory x, time t, and condition c is identical to the 1D Temporal CNN. The input is reshaped and projected

with a 1x1 convolution, and the time and condition embeddings are fused and added to this initial hidden state.

2. **Encoder Path (Downsampling):** The architecture employs two DownBlock modules. Each DownBlock consists of a ResidualBlock followed by a strided Conv1d layer that halves the temporal resolution of the feature map. The output of the residual block is also passed forward as a skip connection to the corresponding decoder block.

3. **Bottleneck:** At the lowest temporal resolution, a single ResidualBlock processes the feature map without further downsampling.

4. **Decoder Path (Upsampling):** The architecture employs two UpBlock modules. Each UpBlock first uses a ConvTranspose1d layer to double the temporal resolution of its input. The upsampled feature map is then concatenated with the corresponding skip connection from the encoder path. This combined feature map is then processed by a ResidualBlock.

5. **Output:** A final 1x1 convolution (final_conv) projects the output of the final UpBlock back to the original transition_dim. The output is then reshaped to match the original flattened trajectory format.

5.1.4 Results and Analysis

The experimental results highlight a consistent set of trade-offs between flow matching and diffusion paradigms across all tested architectures. While both methods prove capable of modeling the trajectory distributions, they exhibit several key differences in efficiency, determinism, and training dynamics.

Table 5.1: Training and inference times for Diffusion vs. Flow Matching across architectures.

Architecture	**Training Time per Epoch (s)**	**Inference Time (s)**
Diffusion MLP	16.68	0.0820
Flow Matching MLP	**16.14**	0.0185
Diffusion CNN	19.59	0.1572
Flow Matching CNN	19.04	0.0448
Diffusion U-Net	26.86	0.2470
Flow Matching U-Net	26.37	**0.0843**

The most significant quantitative difference observed was in inference speed. For a given architecture, Flow Matching was consistently 3-4x faster at generating a single trajectory compared to the standard DDPM sampling schedule. This efficiency gain is a direct result of its formulation; as a deterministic ODE, it can often be solved in fewer steps than a stochastic SDE. While diffusion sampling can be accelerated using schedulers like DDIM, this often comes at the cost of reduced stochasticity and sample fidelity [39].

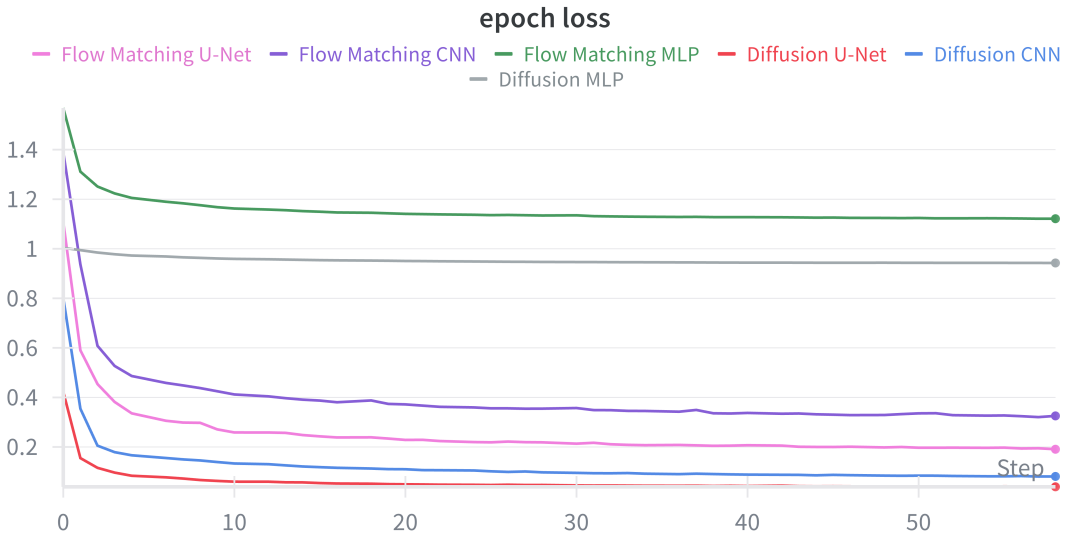

Figure 5.4: Architecture vs. Epoch Loss over 50 epochs

Regarding training, both models had comparable wall-clock training times. While Flow Matching achieved a lower final training loss, this did not directly translate to higher control rewards in the subsequent MPC evaluation. The MPC results for these unconditional models were noisy, but suggested that Flow Matching performed ”less-badly,” likely due to the deterministic and stable nature of its generated trajectories.

Table 5.2: Results and sample efficiency for Diffusion vs. Flow Matching across architectures.

Architecture	**Results**	**Sample Efficiency (loss/epoch)**
Diffusion MLP	-444.92 ± 158.08	3.66e-5
Flow Matching MLP	-283.53 ± 147.83	2.77e-4
Diffusion CNN	-542.90 ± 118.70	3.68e-4
Flow Matching CNN	**-257.14 ± 127.00**	**5.59e-4**
Diffusion U-Net	-312.37 ± 81.43	1.43e-4
Flow Matching U-Net	-320.12 ± 138.21	3.48e-4

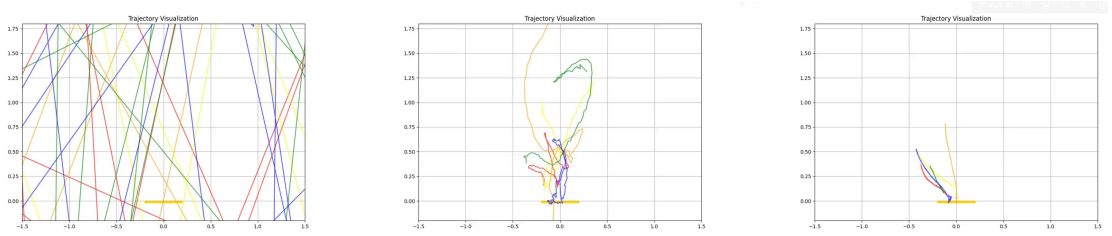

Figure 5.5: Diffusion MLP vs. CNN vs. U-Net

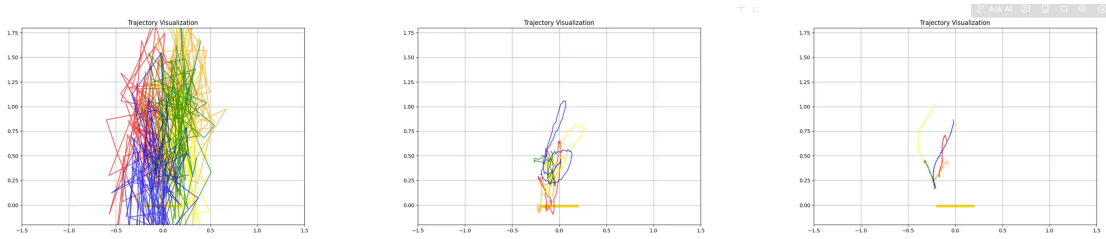

Figure 5.6: Flow Matching MLP vs. CNN vs. U-Net

As anticipated, the more sophisticated U-Net consistently produced trajectories with superior temporal coherence, confirming the benefits of its multi-scale processing for both Flow Matching and Diffusion. The core difference between the methods became visually apparent in the nature of the generated paths.

- Flow Matching consistently produced more direct, deterministic trajectories. For a given starting noise, the resulting path is fixed, reflecting its roots as a learned ODE.

- Diffusion Models, by comparison, generated a much wider variety of "creative" and diverse paths. This is a direct consequence of its stochastic SDE formulation, which allows it to explore a broader range of solutions.

This distinction was also reflected in the training process. Flow Matching models began producing coherent, structured paths very early in training. This can be attributed to its direct regression objective, which provides a somewhat constrained learning signal from the outset. Diffusion models, however, took significantly longer to form coherent trajectories, as the network must first learn the score function across all noise levels before it can effectively reverse the diffusion process. This resulted in "wilder", less structured generation in the early stages of training. From a practical standpoint, the diffusion framework also requires much more "bells and whistles" (e.g. noise schedule $\beta(t)$, training steps N, inference steps K, learning rate schedules $\eta(i)$, making flow matching the simpler paradigm to implement.

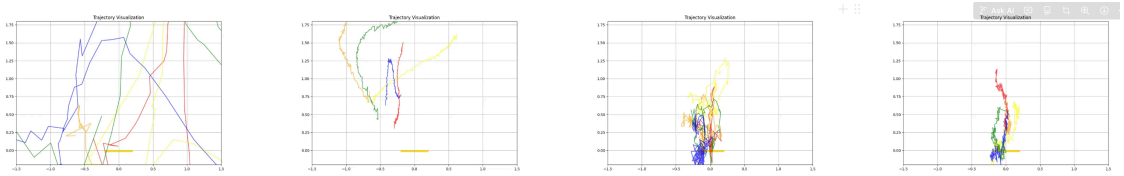

Figure 5.7: Diffusion CNN at Epoch 0 and 5 (left) vs. FM CNN at Epoch 0 and 5 (right)

5.2 Planner Baseline: The Joint Planner

Having identified the U-Net as the most expressive network backbone in the initial analysis, this section evaluates the performance of the monolithic Joint Planner (JP) using this architecture. The goal is to establish a performance baseline for an end-to-end approach under two distinct conditioning schemes, quantifying its capabilities and limitations before evaluating the fully decoupled framework in Section 5.3.

5.2.1 Experimental Setup

For this comparison, both the Flow Matching and Diffusion planners were implemented using the U-Net architecture. To provide a comprehensive analysis of practical inference trade-offs, results are also presented for the Diffusion planner using an accelerated DDIM scheduler [39]. This comparison highlights how sampling can be sped up, typically at the cost of reduced sample fidelity and stochasticity.

The evaluation of these planners centers on three key metrics: final control performance, training time per epoch, and inference time per trajectory. Final control performance is measured by cumulative reward in a closed-loop MPC evaluation, where a score above 100 indicates a successful landing.

5.2.2 Conditioning Signals and Fusion Strategy

Two key design choices were investigated for conditioning the planners: the fusion strategy for combining embeddings and the type of goal signal used. The first choice concerns how the time embedding and the condition embedding are combined. Two fusion strategies were implemented and compared:

1. **Addition:** The embeddings are first projected to the same hidden dimensions then combined via simple element-wise addition. This approach imposes a strong inductive bias, forcing the model to learn time and condition embeddings that exist in the same semantic space where their dimensions are directly comparable.
2. **Concatenation:** The embeddings are first concatenated and then passed through a linear layer to learn a joint representation. This strategy offers a bit more expressive power because the network can learn a complex, non-linear relationship between time and the condition. However, it lacks the strong structural bias the addition method provides.

The second methodological choice involves the goal conditioning signal, comparing the two strategies formulated in Section 3.3.1:

1. **Final Goal Conditioning ($s_{\mathbf{goal}} = s_T$):** The planner is conditioned on the final goal of the entire task. This provides the model with a more global objective.
2. **Waypoint Goal Conditioning ($s_{\mathbf{goal}} = s_H$):** The planner is conditioned on an intermediate goal at the end of its planning horizon. This may encourage the model to generate more coherent local plan segments.

5.2.3 Classifier-free Guidance vs. Modeling Conditional Distribution

A fundamental difference between the Diffusion and Flow Matching planners lies in how they incorporate conditioning. The Diffusion JP employs Classifier-Free Guidance (CFG), a powerful technique for strengthening the conditioning signal at inference time [40]. CFG allows the model to be paused at each denoising step to compute both a conditional and an unconditional prediction. These are then combined to "steer" the generation towards the desired outcome, with the strength of this steering controlled by a guidance scale. In this work, a scale of 1.5 was used to ensure strong adherence to the condition.

This iterative intervention is not naturally compatible with the Flow Matching planner. A Flow Matching model is trained as a deterministic ODE, and inference is a more holistic process where a numerical solver integrates the entire vector field v_θ from start to finish. Attempting to apply the CFG trick at each internal step of an ODE solver would violate its assumptions and be computationally inefficient. While recent work has begun to explore CFG-like techniques for flow models [41], I opted for injecting the condition directly into the network, $v_\theta(x_t, t, c)$. It is worth noting that inference-time flexibility afforded by diffusion's iterative guidance is a distinct advantage over the standard flow-matching paradigm.

5.2.4 Results and Analysis

The results highlight several key findings regarding the monolithic planner's performance and design, as seen in Table 5.3 and 5.4:

Table 5.3: Comparison of Flow Joint Planner (JP) fusion strategies under different goal conditioning methods.

Model	MPC Results	Training Time (s)	Inference Time (s)
Flow JP Goal (concat)	**172.93 ± 38.55**	60.82	**0.0910**
Flow JP Goal (add)	145.09 ± 22.01	63.52	0.0920
Flow JP Waypoint (concat)	136.98 ± 38.14	**42.31**	0.0930
Flow JP Waypoint (add)	143.91 ± 26.35	43.30	0.0932

First, the choice of fusion strategy for the time and condition embeddings had some noticeable impact. For final goal conditioning, the more expressive concatenation strategy

performs best. This suggests that the model benefits from the flexibility of the joint representation. For waypoint goal conditioning, the addition strategy performs better. The strong inductive bias imposed by addition appears to act as a powerful regularizer for simpler, more well-defined tasks.

Table 5.4: Comparison of Diffusion and Flow Joint Planners (JP) under different configurations.

Model	**MPC Results**	**Training Time (s)**	**Inference Time (s)**
Diffusion JP Goal	116.71 ± 55.73	60.84	0.2780
Diffusion JP Waypoint	69.98 ± 85.93	42.70	0.2670
Diffusion JP Goal (DDIM)	116.97 ± 82.78	62.54	0.1244
Flow JP Goal	**172.93 ± 38.55**	60.82	**0.0910**
Flow JP Waypoint	136.98 ± 38.14	**42.30**	0.0930

Regarding the conditioning signals, conditioning on the final task goal proved to be a much more effective strategy than conditioning on an intermediate waypoint for both paradigms. Conditioning on a waypoint led to a substantial performance drop for both models, indicating that for a monolithic planner on this task, global context is critical. This is likely because providing the ultimate objective allows the model to generate a more globally coherent plan, which is particularly effective for a task like Lunar Lander where the task horizon T is reasonably close to the model's planning horizon H.

Finally, the Flow Matching planner significantly outperforms the Diffusion planner in terms of MPC reward, achieving a top score of 172.93 compared to the diffusion baseline of 116.71. This suggests that the deterministic and stable nature of the ODE-based generation is beneficial for this control task. Furthermore, the results confirm the significant inference speed advantage of Flow Matching. The Flow JP generated trajectories approximately 3x faster than the Diffusion JP using a standard DDPM sampler. While a tuned DDIM sampler closes this gap considerably (halving the DDPM inference time), it does so without improving the final reward.

5.3 Flat vs. Hierarchical Planning

While effective, the monolithic ”inpainting” approach for goal-conditioning is computationally expensive because it requires generating a complete state-action trajectory. In contrast, modeling only the conditional action distribution, as seen in Diffusion Policy, creates a fast, reactive policy but often struggles with long-term strategic reasoning. This section evaluates our proposed hierarchical framework that balances these trade-offs by decoupling strategic, long-horizon planning from low-level, reactive control.

The analysis unfolds in two parts. First, we evaluate the individual components (WP and AP) in isolation to validate their standalone capabilities. Second, we assess the full hierarchical system against the baselines established in Section 5.2 and our original expert policy. The Waypoint Planner is designed with a horizon of 100 steps, whereas the Action Planner is trained to predict 25 steps.

5.3.1 Individual Planner Performance

As explained, the Waypoint Planner generates a set of observations only. To evaluate its performance in isolation, the trained WP model was used to generate open-loop state trajectories conditioned on a variety of start and goal states sampled from the expert dataset.

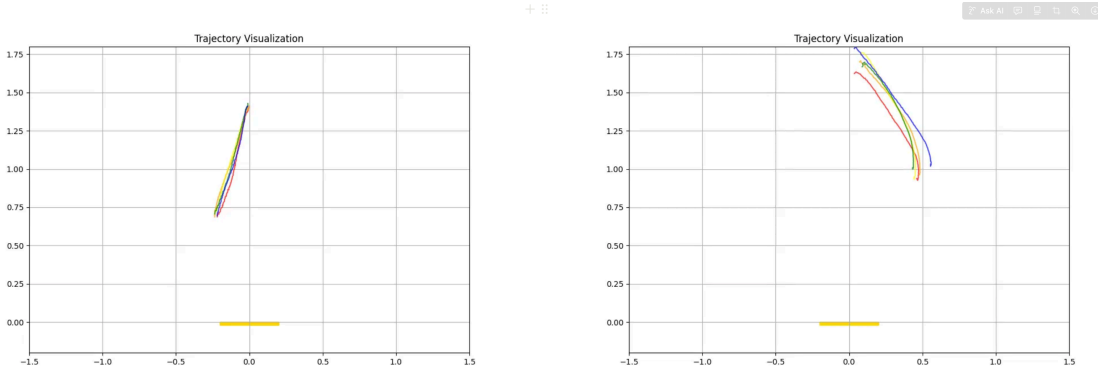

Figure 5.8: Flow vs. Diffusion Waypoint Planner (conditioned on final goal)

A qualitative review of these plans (Figure 5.8) confirms that both waypoint planners successfully generate smooth and dynamically plausible waypoint sequences. The generated paths consistently connect the start states (typically at a height between 1.4 and 1.8) and point to the final landing pad. However, the trajectories generated by the Diffusion

WP show significantly more diversity and variation, while the Flow WP produces more direct and deterministic paths. This contrast reinforces the stochastic, exploratory nature of diffusion versus the deterministic, goal-oriented nature of flow matching.

Next, the Action Planner handles low-level, reactive control. Its performance was first evaluated in an open-loop "tracking" task. The trained AP model received the agent's current state and a waypoint sampled from a ground-truth expert trajectory, then generated an action sequence to reach that waypoint. The results are summarized in the table below.

Table 5.5: Comparison of Diffusion vs. Flow Matching Action Planners (AP).

Model	**Training Time (s)**	**Inference Time (s)**	**MPC**
Diffusion AP	40.28	0.2956	101.27 ± 52.45
Flow AP	**39.38**	**0.0912**	**133.93 ± 30.35**

The results clearly demonstrate the advantages of the flow matching paradigm for reactive control. The Flow AP significantly outperforms its diffusion counterpart, delivering both higher and more consistent rewards. Additionally, it processes inference three times faster. Both AP models benefit from much quicker training times compared to long-horizon planners, as they only need to learn a low-dimensional action distribution over a short time frame.

5.3.2 Hierarchical Planning

The final experiment evaluates the full, integrated Flow Planner framework as described in Section 3.3. The Waypoint Planner generates a strategic plan, which is then executed by the Action Planner in a closed-loop MPC setting. The performance is measured by the cumulative reward over 10 episodes and directly compared to the PPO expert that generated the dataset.

Table 5.6: Amortized inference time across different waypoint (WP) replanning frequencies.

Model	**WP Replan Freq**	**Amortized Inference Time (s)**
PPO Baseline	-	-
Flow WP + Flow AP	10	0.1315
Flow WP + Flow AP	20	0.1118
Flow WP + Flow AP	30	0.0984
Flow WP + Flow AP	40	0.0930
Flow WP + Flow AP	50	**0.0917**

Table 5.7: MPC results and highest reward across different waypoint (WP) replanning frequencies.

Model	**WP Replan Freq**	**MPC Result**	**Highest Reward**
PPO Baseline	-	**236.78** $\pm$ 79.67	**305.92**
Flow WP + Flow AP	10	220.37 $\pm$ 54.30	289.52
Flow WP + Flow AP	20	232.40 $\pm$ **40.78**	295.58
Flow WP + Flow AP	30	210.50 $\pm$ 51.29	278.22
Flow WP + Flow AP	40	194.47 $\pm$ 56.03	274.82
Flow WP + Flow AP	50	180.43 $\pm$ 44.89	256.23

The results from Table 5.6 and 5.7 confirm the effectiveness of the hierarchical Flow Planner framework. The integrated system successfully solves the long-horizon task, achieving a mean reward of 232.40, which is statistically comparable to the original PPO expert's performance. While the mean reward doesn't surpass the expert baseline, the Flow Planner delivers significantly more consistent and reliable performance, as evidenced by its substantially lower standard deviation (40.78 vs. 79.67). This demonstrates that the framework can robustly reproduce the expert policy from the offline dataset!

Furthermore, analysis of the waypoint replan frequency, a tunable hyperparameter, reveals a clear trade-off between strategic guidance and computational cost. Optimal performance occurs when the Waypoint Planner is invoked every 20 steps, suggesting a "sweet spot" that provides sufficient high-level correction without unnecessary overhead.

This configuration also demonstrates the framework's efficiency, achieving an amortized inference time of only 0.1118 seconds per trajectory generation. The planner's highest reward of 295.58 is also competitive with the expert's peak performance, showing the model can indeed generate high-quality, near-optimal trajectories.

6 CONCLUSIONS

The proposed Flow Planner is a generative hierarchical framework that addresses long-horizon decision-making problems by decoupling strategic reasoning from reactive control. This architecture divides planning into two distinct levels: a high-level global planner operating in the observation space to provide long-term strategic direction, and a low-level reactive action planner generating robust, short-horizon policies. This decomposition significantly reduces the complexity of the state space each flow model must learn. Consequently, the action planner can be trained efficiently on short-horizon tasks (e.g., 25 steps) while increasing model capacity (e.g., 64 hidden dimensions $\rightarrow$ 128) to avoid the sample inefficiency and challenges typically associated with end-to-end, long-horizon trajectory planning.

This hierarchical approach yields substantial computational and design benefits. Within an MPC loop, the computationally intensive global planner is invoked infrequently (e.g., once every 20 steps), leading to a low amortized inference time that makes the system practical for real-time applications. Furthermore, the framework's inherent modularity establishes a clear separation of concerns. This allows for independent optimization and debugging of each component: the global planner, the reactive planner, and the controller. This modularity facilitates the integration of hybrid systems that combine the strengths of both learned and classical planning methods.

While the empirical results in this dissertation demonstrated the superiority of Flow Matching for the Lunar Lander task, the framework's design does not advocate for one paradigm over the other. Instead, perhaps the most significant contribution is how the hierarchical structure provides a principled way to select the optimal generative model for different types of planning tasks. For problems that demand strategic exploration across the solution space, where fine-grained controllability and the generation of diverse, stochastic plans are necessary, diffusion models can be a superior paradigm. On the other hand, for tasks where raw sampling speed, deterministic precision, and efficient exploitation are paramount, Flow Matching is a lot more compelling. In this sense, the architecture brings

the discussion full circle, grounding the choice between modern generative models in the classical reinforcement learning tension between exploration and exploitation.

This trade-off is not incidental but is a direct consequence of the models' fundamental formulations. Diffusion models learn a score function to reverse a fixed noising process, making their iterative denoising inference a natural fit for guided, stochastic exploration of the solution space. In contrast, Flow Matching learns a deterministic vector field to directly transport a prior to the data distribution. Its inference is a single ODE integration, which is inherently faster and produces smoother, more direct trajectories, making it the ideal choice for efficient, exploitative control. Ultimately, the choice between these powerful generative tools is a deliberate design decision, and the Flow Planner framework provides a principled architecture for leveraging the distinct strengths of both.

6.1 Evaluation

This work successfully achieved its primary objectives, culminating in the design and validation of a new class of generative planners based on the Flow Matching paradigm. The core technical contribution, presented in Chapter 3, was the novel reformulation of the foundational CFM framework to generate trajectories for robotic control, establishing a new class of deterministic, high-speed generative planners. This formulation was then incorporated into the Flow Planner architecture, a hierarchical framework that divides the planning problem into strategic and reactive components. This hierarchical structure proved essential for overcoming the scaling and long-horizon challenges inherent in monolithic, end-to-end planners. The extensive empirical evaluation in Chapter 5 validated the framework's ability to generate high-quality trajectories that consistently matched the performance of an expert PPO agent. The direct comparison with the diffusion paradigm quantified the significant advantages of the flow-based approach in terms of inference speed and training stability, providing practical guidance for future research. Finally, this work released three comprehensive benchmark datasets along with their corresponding trained expert agents, enabling full reproducibility and providing valuable resources for future research.

6.2 Future Work

6.2.1 Scaling Up to Different Environments

The present work, developed within the Box2D environment, sets a solid foundation for Flow Planner, but its utility will be best demonstrated in more complex domains. Extending experimentation to Gymnasium environments such as Atari and MuJoCo [42], and potentially benchmarking on the latest OG-Bench tasks [cite here], would stress-test the planner's adaptability. Pushing further into realistic robotic arms, using sim-to-real transfer techniques, would offer compelling evidence of robustness and effectiveness. Such scaling efforts would validate the approach's applicability beyond controlled simulations and pave the way toward real-world deployment.

6.2.2 Flow Optimization and other Applications

There are many recent state-of-the-art advancements in optimizing flow for faster inference. For example, rectified flow is a technique that iteratively straightens the generated flow, enabling high-quality generation in a fraction of the ODE steps [43]. Similarly, Optimal Transport Flow (OT-CFM) could improve both training stability and inference speed by learning a more efficient transport map between the noise and data distributions [44]. For tasks involving complex kinematics, such as 6-DoF manipulation, exploring Riemannian Flow Matching would allow the model to respect the underlying manifold geometry of the robot's configuration space (e.g., SO(3)), leading to more natural and physically plausible plans [7].

6.2.3 Strengthening and Diversifying Conditioning Signals

So far, conditioning in Flow Planner has been limited to start and end states, which constrains its expressive potential. A logical next step would be to implement reward-based conditioning, allowing the model to prioritize trajectories with higher expected returns, resulting in more consistent and optimal behavior. Additionally, integrating frameworks like ControlNet could substantially enhance the conditioning mechanism by enabling external guidance signals (e.g. natural language task descriptions, environmental constraints, etc.) to influence trajectory generation. This enhancement would dramatically increase the controller's versatility across diverse tasks with varying requirements.

6.2.4 Integration with Conventional Reinforcement Learning

While Flow Planner has so far focused on imitation-style learning, its architecture is well-suited for integration with conventional reinforcement learning. As mentioned in section 2.2.6 relevant works, Future work should explore incorporating methods such as Q-Score Matching (QSM) or Flow Q-Learning (FQL) to enable direct policy refinement [27, 29]. This would allow the planner to optimize behavior directly according to task-specific reward functions rather than purely reproducing trajectories.

6.2.5 Towards a Hierarchical Reasoning Model

The current Flow Planner architecture mirrors the dual-process "System 1" (fast, reactive) and "System 2" (slow, deliberative) thinking popularized by Kahneman. However, it lacks a crucial feedback loop from the low-level planner back to the high-level planner. A powerful future direction would be to implement an "iterative refinement loop," inspired by the recent Hierarchical Reasoning Model [45] that demonstrated exceptional performance on the ARC-AGI benchmark. This approach would create a feedback loop between global and local planners, allowing the low-level planner to evaluate feasibility and projected costs before executing on the strategic "imaginations". This is a mechanism shown to be critical for performance on complex, multi-step problems.

BIBLIOGRAPHY

[1] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems," Nov. 2020.

[2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," Jun. 2023.

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," Dec. 2020.

[4] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with Diffusion for Flexible Behavior Synthesis," Dec. 2022.

[5] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," Feb. 2023.

[6] P. Holderrieth and E. Erives, "An Introduction to Flow Matching and Diffusion Models," Jun. 2025.

[7] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat, "Flow Matching Guide and Code," Dec. 2024.

[8] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[9] O. G. Younis, R. Perez-Vicente, J. U. Balis, W. Dudley, A. Davey, and J. K. Terry, "Minari," Sep. 2024. [Online]. Available: https://doi.org/10.5281/zenodo.13767625

[10] A. Kumar, J. Fu, G. Tucker, and S. Levine, "Stabilizing Off-Policy Q-Learning via Bootstrapping Error Reduction," Nov. 2019.

[11] Y. Wu, G. Tucker, and O. Nachum, "Behavior Regularized Offline Reinforcement Learning," Nov. 2019.

[12] A. Nair, A. Gupta, M. Dalal, and S. Levine, "AWAC: Accelerating Online Reinforcement Learning with Offline Datasets," Apr. 2021.

[13] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Zou, S. Levine, C. Finn, and T. Ma, "MOPO: Model-based Offline Policy Optimization," Nov. 2020.

[14] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to Trust Your Model: Model-Based Policy Optimization," Nov. 2021.

[15] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-Learning for Offline Reinforcement Learning," Aug. 2020.

[16] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4rl: Datasets for deep data-driven reinforcement learning," 2020.

[17] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," Dec. 2022.

[18] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," Nov. 2015.

[19] P. Vincent, "A Connection Between Score Matching and Denoising Autoencoders," *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, Jul. 2011.

[20] Y. Song and S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," Oct. 2020.

[21] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations," Feb. 2021.

[22] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan, "Normalizing Flows for Probabilistic Modeling and Inference."

[23] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using Real NVP," Feb. 2017.

[24] D. P. Kingma and P. Dhariwal, "Glow: Generative Flow with Invertible 1x1 Convolutions," Jul. 2018.

[25] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural Ordinary Differential Equations," Dec. 2019.

[26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Dec. 2015.

[27] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, "Learning a Diffusion Model Policy from Rewards via Q-Score Matching," Feb. 2025.

[28] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, "Steering Your Diffusion Policy with Latent Space Reinforcement Learning," Jun. 2025.

[29] S. Park, Q. Li, and S. Levine, "Flow Q-Learning," May 2025.

[30] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," Mar. 2015.

[31] A. Tong, N. Malkin, K. Fatras, L. Atanackovic, Y. Zhang, G. Huguet, G. Wolf, and Y. Bengio, "Simulation-free Schrödinger bridges via score and flow matching," Mar. 2024.

[32] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, Jun. 2000.

[33] D. Ha and J. Schmidhuber, "World Models," Mar. 2018.

[34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," Aug. 2017.

[35] A. Raffin, "Rl baselines3 zoo," https://github.com/DLR-RM/rl-baselines3-zoo, 2020.

[36] K. O'Shea and R. Nash, "An Introduction to Convolutional Neural Networks," Dec. 2015.

[37] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," May 2015.

[38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," Aug. 2023.

[39] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," Oct. 2022.

[40] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," Jul. 2022.

[41] W. Fan, A. Y. Zheng, R. A. Yeh, and Z. Liu, "CFG-Zero*: Improved Classifier-Free Guidance for Flow Matching Models," Apr. 2025.

[42] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.

[43] X. Liu, C. Gong, and Q. Liu, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow," Sep. 2022.

[44] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio, "Improving and generalizing flow-based generative models with minibatch optimal transport," Mar. 2024.

[45] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. A. Yadkori, "Hierarchical Reasoning Model," Aug. 2025.

A APPENDIX

A.1 Specializing the SDE for DDPM

Section 2.3 introduced the general SDE framework. This appendix provides the specific forms of these SDEs for the DDPMs models, which are defined by a noise schedule $\beta(t)$.

A.1.1 DDPM Forward SDE

The general forward SDE is given by:

$$dx = f(x,t)dt + g(t)dw \tag{A.1}$$

For the popular Variance Preserving (VP) SDE, which is the continuous-time equivalent of the DDPM forward process, the coefficients are chosen as follows:

- **Drift Coefficient:**

  $$f(x,t) = -\tfrac{1}{2}\beta(t)x$$

  This is a linear drift term that deterministically pulls any data point x towards the origin (the mean of the noise distribution). The rate at which it pulls is controlled by the noise schedule $\beta(t)$.

- **Diffusion Coefficient:**

  $$g(t) = \sqrt{\beta(t)}$$

  This term controls the magnitude of the random noise injected into the process at time t. The amount of noise is also directly governed by the noise schedule $\beta(t)$.

Substituting these specific functions into the general SDE yields the precise form for the DDPM forward process:

$$dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t)}dw \tag{A.2}$$

A.1.2 DDPM Reverse SDE

The corresponding general reverse-time SDE, which transforms noise back into data, is defined as:

$$dx = \left[f(x,t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt + g(t) d\bar{w} \tag{A.3}$$

$$dx = [-\frac{1}{2}\beta(t)x - \beta(t)\nabla_x \log p_t(x)]dt + \sqrt{\beta(t)}d\bar{w} \tag{A.4}$$

This is the equation that is solved during the sampling process. The score function allows the model to deterministically drift the samples towards regions of high data density while being guided by the stochastic diffusion term.

A.2 Derivation of the CFM Objective

The Flow Matching framework is built upon a distinction between marginal and conditional formulations. The marginal quantities represent the true flow to be learned but are computationally intractable. The conditional quantities, however, are simple and tractable. The main takeaway is that a model trained on the tractable conditional objective provably converges to the desired marginal model.

A.2.1 Marginal Formulations

The marginal formulations describe the flow of transforming the noise distribution into the entire data distribution p_data by averaging over all possible data points z.

1. **Marginal Probability Path** p_t: This describes the distribution of points at time t as it morphs from simple noise into the full data distribution, defined by an integral over all possible conditional paths:

$$p_t(x) = \int p_t(x|z) p_{data}(z) dz. \tag{A.5}$$

   Because p_{data} is complex, this integral is intractable.

2. **Marginal Vector Field** $u_t^{\textbf{target}}(x)$: This is the average vector field that generates the marginal path and is the target for the neural network to learn. It is also defined by an intractable integral:

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz \tag{A.6}$$

A.2.2 Conditional Formulations

To solve the intractability of the marginal formulations, simpler conditional quantities are defined with respect to a single point zp_{data}. These describe the process of transforming noise into that one chosen sample and are analytically tractable.

1. **Conditional Probability Path** $p_t(\cdot \mid z)$: This describes a path of probability distributions that interpolates between the initial noise distribution p_{init} and a single data point z. For the Gaussian path used in CFM, the position on this path at time t is defined by the noise schedulers α_t and β_t:

$$x_t = \alpha_t z + \beta_t x_0 \tag{A.7}$$

   The distribution at time t is therefore

$$p_t(x|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d) \tag{A.8}$$

2. **Conditional Vector Field** $u_t^{target}(x \mid z)$: This is the ground truth velocity field that generates the conditional probability path. Taking the derivative of the path equation with respect to time gives the velocity in terms of the start and end points:

$$\text{velocity} = \frac{d}{dt}(\alpha_t z + \beta_t x_0) = \dot{\alpha}_t + \dot{\beta}_t x_0 \tag{A.9}$$

   but the neural network only has access to the current position on the path x, not the initial noise x_0. We can solve x_0 by rearranging the path equation:

$$x_0 = \frac{x - \alpha_t z}{\beta_t} \tag{A.10}$$

   Substituting this back into the velocity equation yields the final vector field formula in terms of the current position x and the target data point z:

$$u_t^{target}(x \mid z) = \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\beta_t \dot{\beta}_t}{\alpha_t}\right) z + \frac{\beta_t \dot{\beta}_t}{\alpha_t} x \tag{A.11}$$

   This tractable field serves as the direct regression target for the neural network.

A.2.3 The Conditional Flow Matching Solution

As shown in Section 3.1, the general Flow Matching objective is to minimize the L2 distance between the learned model v_θ and the intractable marginal vector field $u_t^{target}(x)$. Using the "marginalization trick", one can prove that loss is equivalent (up to a constant) to minimizing the loss with respect to the tractable conditional vector field $u_t^{target}(x \mid z)$.

A.3 Joint Training Objective for Hierarchical Planners

Training a monolithic joint flow, despite its benefits, also introduces new potential failure modes, primarily error propagation. Poorly generated waypoints can lead the action planner astray and inconsistencies may arise if the two components are not well-aligned. These challenges can theoretically be mitigated through joint training. As established in Section 3.3.2, the chain rule of probability provides a way to factorize the joint trajectory distribution. Calculating the total log-likelihood of the data is then equivalent to maximizing the sum of the log-likelihoods of the two hierarchical components:

$$\log p(\tau_s, \tau_a \mid c) = \log p(\tau_a \mid \tau_s, c) + \log p(\tau_s \mid c). \tag{A.12}$$

where λ is a weighting hyperparameters to balance the two terms. To further enforce consistency, this joint objective can be augmented with auxiliary losses such as a reachability penalty. This penalty uses a learned one-step dynamics model $f_\phi(s_k, a_k)$ to predict the state that would be reached if the Action Planner were to execute its plan [30]. The loss that penalizes the distance between the predicted state and the waypoint intended can be formulate as:

$$\mathcal{L}_{\text{reach}}(\theta_{WP}, \theta_{AP}) = \mathbb{E}_{\tau_s^{\text{gen}}} \left[\sum_{k=0}^{H-1} \| f_\phi(s_k^{\text{gen}}, a_k^{\text{gen}}) - s_{k+1}^{\text{gen}} \|^2 \right], \tag{A.13}$$

where s_k^{gen} is a waypoint generated by the WP, and a_k^{gen} is the action generated by the AP to reach the next waypoint. Such a penalty is designed to penalized waypoint transitions that are dissimilar to state transitions covered in the expert dataset, thereby grounding the high-level plan in physically plausible behaviors.

The factorized joint training preserves end-to-end coherence through a single scalar objective while maintaining modularity and disentanglement. Specifically, the state flow can be trained on unlabeled trajectories, reused across different low-level controllers, or

even swapped when transferring to new embodiment. In contrast, a flat joint flow offers maximal expressivity but no such modularity, and its latent representation entangles state and action dynamics that lacks interpretability.

A.4 Latent Dataset Generation

The high-dimensional visual observation spaces of the CarRacing-v3 environment required a specialized data processing pipeline to make the planning problem tractable. This involved training a Variational Autoencoder (VAE) to learn a compressed latent representation of the environment's visual states.

A.4.1 VAE Training

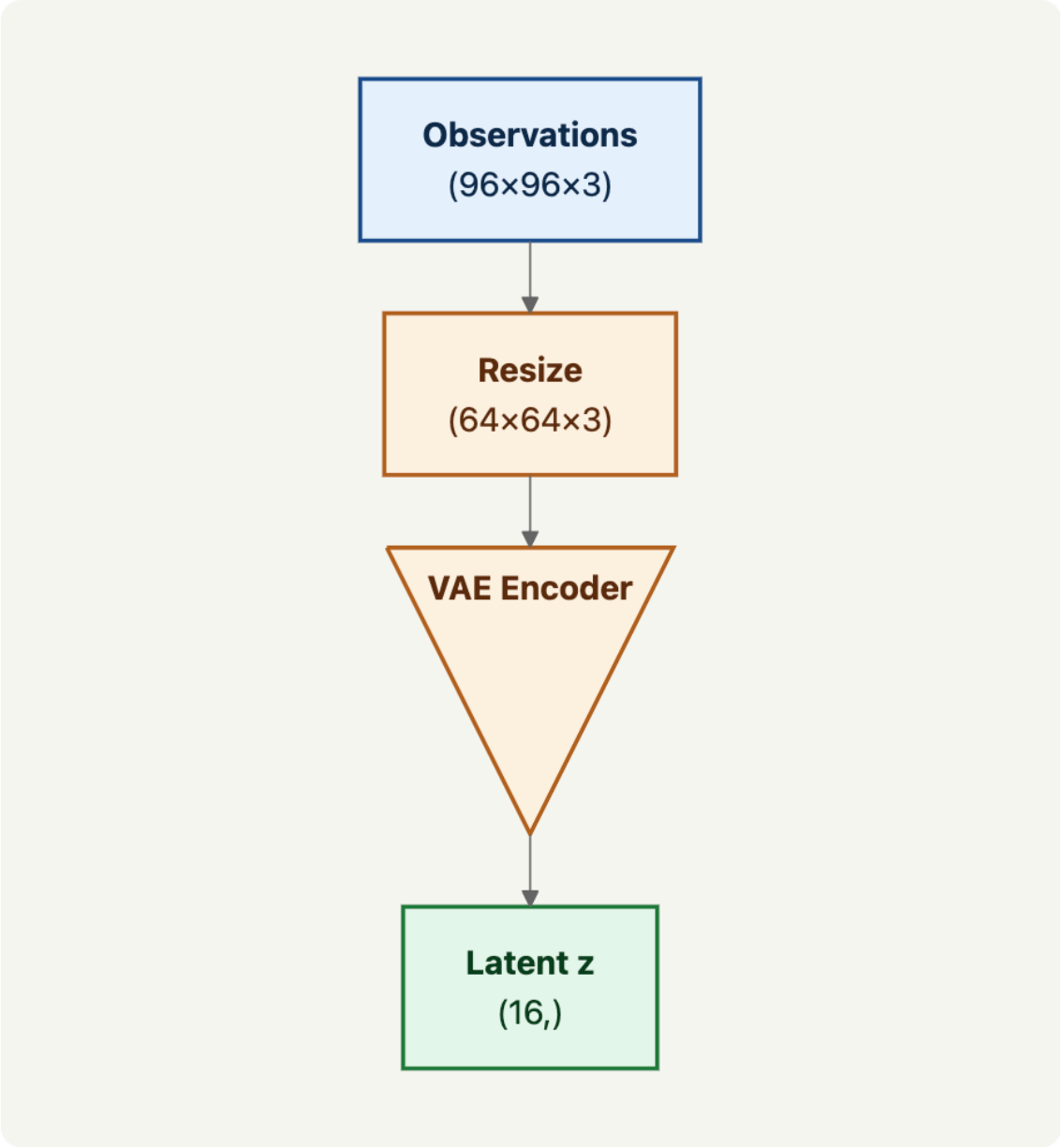

Figure A.1: VAE Pipeline

A.4.2 Latent Dataset and Inference Wrapper

Following the training of the VAE, its encoder was used to process the entire expert PPO dataset. Each image observation in the expert trajectories was passed through the trained encoder to generate a corresponding latent vector. This resulted in a new dataset

containing sequences of `(latent state, action)` pairs, which was then used to train the Flow Planner models.

A.4.3 VAE Training

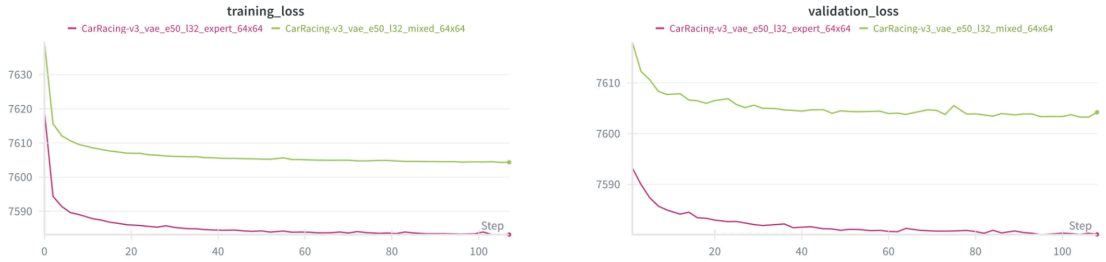

Figure A.2: VAE Loss Curves

A.4.4 VAE Training

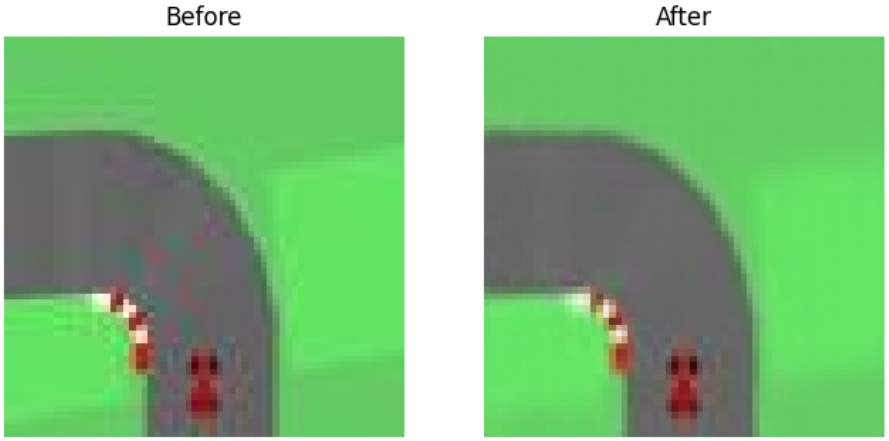

Figure A.3: VAE Reconstruction Results

To facilitate inference, a custom Gymnasium wrapper was created. This wrapper integrates the trained VAE encoder so the planner can operate in real-time by automatically encoding the visual observations from the environment into the latent space.